

문제 01

축소 정복을 가장 직접적으로 나타내는 구조는?

- 1 항상 두 부분 문제를 모두 푼다
- 2 모든 중간 결과를 반드시 표에 저장한다
- 3 가능한 모든 완성 답을 생성한다
- 4 현재 가장 이익이 큰 선택만 확정한다
- 5 더 작은 한 부분 문제를 풀고 원래 답을 구성한다

문제 01 정답과 해설

축소 정복을 가장 직접적으로 나타내는 구조는?

5번 더 작은 한 부분 문제를 풀고 원래 답을 구성한다

해설

핵심은 같은 유형의 더 작은 한 문제로 이어지는 구조다. 재귀나 표의 사용 여부만으로 분류하지 않는다.

문제 02

입력 크기 n 은 0 이상의 정수이고, $T(n)$ 은 전체 기본 연산 횟수이다. 크기가 0이면 연산 1회로 끝낸다. 그 외에는 크기 $n-1$ 의 문제를 한 번 풀고 연산을 1회 더 한다. 따라서 $T(0)=1$, $T(n)=T(n-1)+1$ 이다. $T(n)$ 의 증가 정도를 가장 정확하게 나타내는 것은?

- 1 $\Theta(1)$
- 2 $\Theta(n^2)$
- 3 $\Theta(n)$
- 4 $\Theta(n \log n)$
- 5 $\Theta(\log n)$

문제 02 정답과 해설

입력 크기 n 은 0 이상의 정수이고, $T(n)$ 은 전체 기본 연산 횟수이다. 크기가 0이면 연산 1회로 끝낸다. 그 외에는 크기 $n-1$ 의 문제를 한 번 풀고 연산을 1회 더 한다. 따라서 $T(0)=1$, $T(n)=T(n-1)+1$ 이다. $T(n)$ 의 증가 정도를 가장 정확하게 나타내는 것은?

3번 $\Theta(n)$

해설

n 을 1씩 줄이는 단계가 n 번이고, 각 단계에서 연산 1회와 마지막 종료 연산 1회가 필요하다. $T(n)=n+1$ 이므로 $\Theta(n)$ 이다.

문제 03

입력 크기 n 은 1,2,4,8,... 중 하나이고, $T(n)$ 은 전체 기본 연산 횟수이다. 크기 1이면 연산 1회로 끝낸다. 크기가 $m > 1$ 인 단계에서는 연산 m 회를 하고, 크기 $m/2$ 의 문제 하나만 계속 푼다. 즉 $T(1)=1$, $T(n)=T(n/2)+n$ 이다. 전체 연산 횟수의 가장 정확한 증가 정도는?

- 1 $\Theta(2^n)$
- 2 $\Theta(n^2)$
- 3 $\Theta(\log n)$
- 4 $\Theta(n)$
- 5 $\Theta(n \log n)$

문제 03 정답과 해설

입력 크기 n 은 1,2,4,8,... 중 하나이고, $T(n)$ 은 전체 기본 연산 횟수이다. 크기 1이면 연산 1회로 끝낸다. 크기가 $m > 1$ 인 단계에서는 연산 m 회를 하고, 크기 $m/2$ 의 문제 하나만 계속 푼다. 즉 $T(1)=1$, $T(n)=T(n/2)+n$ 이다. 전체 연산 횟수의 가장 정확한 증가 정도는?

4번 $\Theta(n)$

해설

단계별 연산을 더하면 $n+n/2+n/4+\dots+1=2n-1$ 이다. 단계 수만 세면 로그지만 전체 연산 횟수는 n 에 비례한다.

문제 04

재귀 함수의 종료를 보장하는 설명으로 충분한 것은?

- 1 멈추는 입력에서는 바로 반환하고, 그 외에는 0 이상의 정수로 켜 문제 크기가 매번 줄어든다
 - 2 호출할 때마다 인수가 증가하면 반드시 종료한다
 - 3 입력의 절댓값이 감소하지 않아도 인수 하나가 바뀌면 된다
 - 4 멈추는 조건이 있으면 그 조건에 도달하지 않아도 된다
 - 5 매 호출에서 같은 크기의 새 배열을 만들면 된다
-
-

문제 04 정답과 해설

재귀 함수의 종료를 보장하는 설명으로 충분한 것은?

1번 멈추는 입력에서는 바로 반환하고, 그 외에는 0 이상의 정수로 잰 문제 크기가 매번 줄어든다

해설

멈추는 조건과 그 조건에 도달하도록 문제 크기가 줄어드는지를 함께 확인한다.

문제 05

$n!$ 을 n 부터 1까지 곱해 구한다. 재귀 구현은 n 을 1씩 줄이고 반복 구현도 n 회 안팎 곱한다. 기본 연산을 $O(1)$ 로 셀 때 옳은 설명은?

- 1 재귀 구현만 $\Theta(\log n)$ 시간이다
- 2 반복 구현은 항상 $\Theta(n^2)$ 시간이다
- 3 재귀 구현은 곱셈을 한 번만 한다
- 4 둘 다 $\Theta(n)$ 시간에 같은 값을 계산한다
- 5 두 구현은 서로 다른 팩토리얼 값을 계산한다

문제 05 정답과 해설

$n!$ 을 n 부터 1까지 곱해 구한다. 재귀 구현은 n 을 1씩 줄이고 반복 구현도 n 회 안팎 곱한다. 기본 연산을 $O(1)$ 로 셀 때 옳은 설명은?

4번 둘 다 $\Theta(n)$ 시간에 같은 값을 계산한다

해설

두 구현 모두 n 에 비례하는 횟수만큼 곱한다. 재귀를 썼다는 이유만으로 더 빠르거나 다른 결과가 나오지는 않는다.

문제 06

다음 Python 함수는 n 번째 피보나치 수를 구한다. n 은 0 이상의 정수이며, $\text{fib}(0)=0$, $\text{fib}(1)=1$ 이고 이후 항은 앞의 두 항을 더한 값이다. 계산 결과를 따로 저장하지 않는다.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

n 이 커질수록 같은 피보나치 수를 반복문으로 차례로 구하는 방식보다 계산량이 빠르게 늘어나는 주된 이유는?

- | | | | |
|---|--|---|--|
| 1 | 같은 인수의 부분 문제를 여러 번 계산한다 | 4 | 더 호출하지 않고 끝내는 입력이 두 가지이면 호출 깊이가 항상 2가 된다 |
| 2 | 한 단계에서 두 값을 더하면 덧셈 횟수와 무관하게 지수 시간이 된다 | 5 | 재귀 호출 깊이만큼만 덧셈을 하므로 중복이 없다 |
| 3 | $\text{fib}(n-2)$ 는 $\text{fib}(n-1)$ 의 계산에 포함되지 않으므로 독립이다 | | |

문제 06 정답과 해설

다음 Python 함수는 n 번째 피보나치 수를 구한다. n 은 0 이상의 정수이며, $\text{fib}(0)=0$, $\text{fib}(1)=1$ 이고 이후 항은 앞의 두 항을 더한 값이다. 계산 결과를 따로 저장하지 않는다.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

n 이 커질수록 같은 피보나치 수를 반복문으로 차례로 구하는 방식보다 계산량이 빠르게 늘어나는 주된 이유는?

1번 같은 인수의 부분 문제를 여러 번 계산한다.

해설

예를 들어 $\text{fib}(4)$ 는 $\text{fib}(3)$ 과 $\text{fib}(2)$ 를 호출한다. $\text{fib}(3)$ 안에서도 $\text{fib}(2)$ 를 다시 호출하므로 같은 값을 두 번 계산한다. 결과를 저장하지 않아 n 이 커질수록 이런 중복 호출이 많아진다.

문제 07

Python의 피보나치 반복에서 $a, b = b, a + b$ 를 수행한다. 이 다중 대입의 올바른 해석은?

- 1 한 번에 두 항을 건너뛰다
 - 2 a 와 b 가 항상 같은 값이 된다
 - 3 b 만 변한다
 - 4 a 를 먼저 바꾼 뒤 바뀐 a 로 b 를 계산한다
 - 5 오른쪽의 두 식을 갱신 전 a, b 로 계산한다
-
-

문제 07 정답과 해설

Python의 피보나치 반복에서 $a, b = b, a + b$ 를 수행한다. 이 다중 대입의 올바른 해석은?

5번 오른쪽의 두 식을 갱신 전 a, b 로 계산한다

해설

Python의 다중 대입은 오른쪽 값을 먼저 평가한다. C로 옮길 때는 임시 변수가 필요하다.

문제 08

한 번의 재귀 호출로 크기 $N/3$ 의 별무늬를 구해 재사용한다. 최종 결과는 N 행 N 열이며 별과 공백을 합쳐 N^2 개 문자를 출력한다. 문자 하나 출력에 일정한 시간이 들 때, 무늬를 계산하는 시간은 제외하고 출력 작업 자체의 가장 정확한 시간 복잡도는?

- 1 $\Theta(N^2)$
- 2 $\Theta(N \log N)$
- 3 $\Theta(\log N)$
- 4 $\Theta(N)$
- 5 $\Theta(1)$

문제 08 정답과 해설

한 번의 재귀 호출로 크기 $N/3$ 의 별무늬를 구해 재사용한다. 최종 결과는 N 행 N 열이며 별과 공백을 합쳐 N^2 개 문자를 출력한다. 문자 하나 출력에 일정한 시간이 들 때, 무늬를 계산하는 시간은 제외하고 출력 작업 자체의 가장 정확한 시간 복잡도는?

1번 $\Theta(N^2)$

해설

출력 문자 자체가 N^2 개이므로 깊이가 로그여도 전체 출력 시간은 최소 이차다.

문제 09

n 개 원소를 왼쪽부터 하나씩 확인하는 순차 탐색에서 찾는 값이 없을 때, 배열 원소와 목표값의 일치 비교는 총 몇 회인가? 반복 조건 검사는 세지 않는다.

- 1 $\lceil \log_2 n \rceil$
 - 2 $n-1$
 - 3 $n(n-1)/2$
 - 4 n
 - 5 1
-
-

문제 09 정답과 해설

n 개 원소를 왼쪽부터 하나씩 확인하는 순차 탐색에서 찾는 값이 없을 때, 배열 원소와 목표값의 일치 비교는 총 몇 회인가? 반복 조건 검사는 세지 않는다.

4번 n

해설

찾는 값이 없으면 모든 원소를 확인해야 한다.

문제 10

정렬되지 않은 배열에도 정확하게 적용할 수 있는 탐색은?

- 1 정렬 없이 제공된 간격의 값만 검사하는 탐색
- 2 순차 탐색
- 3 정렬 없이 중간 원소와 비교하여 한쪽을 버리는 이진 탐색
- 4 정렬 없이 끝값 비율로 위치를 추정해 한쪽을 버리는 보간 탐색
- 5 정렬 없이 첫 원소보다 작으면 즉시 실패하는 탐색

문제 10 정답과 해설

정렬되지 않은 배열에도 정확하게 적용할 수 있는 탐색은?

2번 순차 탐색

해설

순차 탐색은 순서에 의존하지 않고 각 원소를 직접 확인한다.

문제 11

오름차순 배열 a 에서 목표값 x 를 이진 탐색한다. lo 는 현재 후보 구간의 첫 인덱스, hi 는 마지막 인덱스이며 양 끝을 모두 포함한다. 가운데 인덱스를 mid 라고 할 때 $a[mid] < x$ 이다. 다음 후보 구간을 정하는 올바른 갱신은?

- 1 $hi = mid$
- 2 $lo = mid$
- 3 $hi = mid - 1$
- 4 $lo = hi + 1$
- 5 $lo = mid + 1$

문제 11 정답과 해설

오름차순 배열 a 에서 목표값 x 를 이진 탐색한다. lo 는 현재 후보 구간의 첫 인덱스, hi 는 마지막 인덱스이며 양 끝을 모두 포함한다. 가운데 인덱스를 mid 라고 할 때 $a[mid] < x$ 이다. 다음 후보 구간을 정하는 올바른 갱신은?

5번 $lo = mid + 1$

해설

mid 와 그 왼쪽은 모두 x 보다 작아 후보에서 제외된다.

문제 12

오름차순 배열에서 목표값을 이진 탐색한다. lo 는 후보 구간의 첫 인덱스, hi 는 마지막 인덱스이고 양 끝을 모두 포함한다. 탐색 중 $lo > hi$ 가 되어 후보가 남지 않았다. 이것은 무엇을 뜻하는가?

- 1 반드시 $a[lo]$ 가 정답이다
- 2 중간 인덱스를 한 번 더 검사해야 한다
- 3 배열이 비어 있었다는 뜻만 된다
- 4 찾는 값이 배열에 없다
- 5 원소가 중복된다는 뜻이다

문제 12 정답과 해설

오름차순 배열에서 목표값을 이진 탐색한다. lo 는 후보 구간의 첫 인덱스, hi 는 마지막 인덱스이고 양 끝을 모두 포함한다. 탐색 중 $lo > hi$ 가 되어 후보가 남지 않았다. 이것은 무엇을 뜻하는가?

4번 찾는 값이 배열에 없다

해설

유지한 후보 집합이 공집합이므로 실패로 종료한다.

문제 13

오름차순 배열에 같은 값이 여러 번 있고, 찾는 값도 배열에 존재한다. 중간값이 일치하면 즉시 반환하는 이진 탐색이 보장하는 것은?

- 1 항상 첫 인덱스를 반환한다
 - 2 일치값의 개수를 반환한다
 - 3 일치하는 인덱스 하나를 반환한다
 - 4 중복값이 있으면 항상 실패한다
 - 5 항상 마지막 인덱스를 반환한다
-
-

문제 13 정답과 해설

오름차순 배열에 같은 값이 여러 번 있고, 찾는 값도 배열에 존재한다. 중간값이 일치하면 즉시 반환하는 이진 탐색이 보장하는 것은?

3번 일치하는 인덱스 하나를 반환한다

해설

목표값과 일치한 mid를 바로 반환하므로, 왼쪽이나 오른쪽에 같은 값이 더 있는지는 확인하지 않는다.

문제 16

보간 탐색의 위치 추정이 이진 탐색과 다른 점은?

- 1 정렬 여부를 무시한다
- 2 항상 가운데만 확인한다
- 3 끝점의 값과 목표값을 사용한다
- 4 항상 첫 원소만 확인한다
- 5 모든 후보의 합을 계산한다

문제 16 정답과 해설

보간 탐색의 위치 추정이 이진 탐색과 다른 점은?

3번 끝점의 값과 목표값을 사용한다

해설

보간 탐색은 값이 퍼진 정도를 이용해 예상 위치를 계산한다.

문제 17

오름차순 배열 a 의 양 끝을 포함한 후보 구간은 $[lo, hi]$ 이고, 찾는 값은 x 이다. $lo \leq hi$ 이며 인덱스는 0부터 센다.

보간 탐색의 위치 추정식은 다음과 같다. $//$ 는 정수 몫이다.

$$pos = lo + ((x - a[lo]) * (hi - lo)) // (a[hi] - a[lo])$$

그런데 $a[lo] = a[hi]$ 이다. 이 식을 계산하기 전에 어떤 처리가 필요한가?

- 1 목표값과 무관하게 실패를 반환한다
- 2 분모를 1로 바꿔 기존 위치 공식을 그대로 사용한다
- 3 목표값과 무관하게 lo 를 반환한다
- 4 구간 길이가 1이 될 때까지 같은 나눗셈 공식을 반복한다
- 5 목표값과 비교해 성공 또는 실패로 종료한다

문제 17 정답과 해설

오름차순 배열 a 의 양 끝을 포함한 후보 구간은 $[lo, hi]$ 이고, 찾는 값은 x 이다. $lo \leq hi$ 이며 인덱스는 0부터 센다.

보간 탐색의 위치 추정식은 다음과 같다. $//$ 는 정수 몫이다.

$$pos = lo + ((x - a[lo]) * (hi - lo)) // (a[hi] - a[lo])$$

그런데 $a[lo] = a[hi]$ 이다. 이 식을 계산하기 전에 어떤 처리가 필요한가?

5번 목표값과 비교해 성공 또는 실패로 종료한다

해설

분모가 0이다. 정렬된 구간의 양 끝 값이 같으면 구간 전체 값도 같다.

문제 18

보간 탐색이 잘 맞는 입력의 특징은?

- 1 길이는 크지만 값 순서가 무작위인 배열이다
- 2 정렬되어 있고 값이 비교적 균등하게 분포한다
- 3 정렬되어 있지만 거의 모든 값이 한쪽 끝에 몰려 있다
- 4 인덱스 간격은 일정하지만 값의 분포는 고려하지 않는다
- 5 입력 길이가 2의 거듭제곱이면 값의 분포와 무관하다

문제 18 정답과 해설

보간 탐색이 잘 맞는 입력의 특징은?

2번 정렬되어 있고 값이 비교적 균등하게 분포한다

해설

평균적인 효율은 값 분포 가정에 영향을 받으며 최악 선형 시간 가능성은 남는다.

문제 19

길이 n 인 오름차순 숫자 배열에서 보간 탐색으로 목표값을 찾는다. 배열 값의 분포가 고르다는 가정은 없고, 원소 한 번의 접근·비교·기본 산술 연산은 상수 시간으로 센다. 이 탐색의 최악 시간 복잡도를 가장 정확히 나타내는 것은?

- 1 $\Theta(n)$
- 2 $\Theta(n^2)$
- 3 $\Theta(\log n)$
- 4 $\Theta(\log \log n)$
- 5 $\Theta(n \log n)$

문제 19 정답과 해설

길이 n 인 오름차순 숫자 배열에서 보간 탐색으로 목표값을 찾는다. 배열 값의 분포가 고르다는 가정은 없고, 원소 한 번의 접근·비교·기본 산술 연산은 상수 시간으로 센다. 이 탐색의 최악 시간 복잡도를 가장 정확히 나타내는 것은?

1번 $\Theta(n)$

해설

값 분포가 치우치면 조금씩만 후보가 줄어들 수 있다.

문제 20

n 개 원소($n \geq 2$)가 있는 고정 배열에서 값을 찾는 요청을 m 번($m \geq 1$) 처리한다. 처음 한 번만 배열을 정렬하며, 정렬 시간은 $\Theta(n \log n)$ 이라고 주어진다. 이후 각 요청은 이진 탐색으로 처리한다. 배열은 중간에 바뀌지 않는다. 정렬과 m 번 탐색을 합친 최악의 총 시간 복잡도는?

- 1 $\Theta(n \log m)$
- 2 $\Theta(m+n)$
- 3 $\Theta(nm \log n)$ 만 가능
- 4 $\Theta(n \log n + m \log n)$
- 5 $\Theta(\log(n+m))$

문제 20 정답과 해설

n 개 원소($n \geq 2$)가 있는 고정 배열에서 값을 찾는 요청을 m 번($m \geq 1$) 처리한다. 처음 한 번만 배열을 정렬하며, 정렬 시간은 $\Theta(n \log n)$ 이라고 주어진다. 이후 각 요청은 이진 탐색으로 처리한다. 배열은 중간에 바뀌지 않는다. 정렬과 m 번 탐색을 합친 최악의 총 시간 복잡도는?

4번 $\Theta(n \log n + m \log n)$

해설

최악 비용 기준: 정렬 1회의 $\Theta(n \log n)$ 과 탐색 m 회의 $\Theta(m \log n)$ 을 더한다. 검색할 때마다 다시 정렬하지 않는다.

문제 21

$a \geq 0$, $b > 0$ 인 정수에서 $\gcd$ 는 최대공약수, $a \bmod b$ 는 a 를 b 로 나눈 나머지이다. 유클리드 알고리즘의 $\gcd(a,b) = \gcd(b, a \bmod b)$ 가 성립하는 이유는?

- 1 두 수의 합이 보존되기 때문이다
 - 2 두 쌍의 공약수 집합이 같다
 - 3 나머지는 항상 1이기 때문이다
 - 4 큰 수가 매번 정확히 절반이 되기 때문이다
 - 5 두 수의 곱이 보존되기 때문이다
-
-

문제 21 정답과 해설

$a \geq 0, b > 0$ 인 정수에서 $\gcd$ 는 최대공약수, $a \bmod b$ 는 a 를 b 로 나눈 나머지이다. 유클리드 알고리즘의 $\gcd(a,b) = \gcd(b, a \bmod b)$ 가 성립하는 이유는?

2번 두 쌍의 공약수 집합이 같다

해설

$a = qb + r$ 관계를 이용하면 a, b 의 공약수와 b, r 의 공약수가 서로 일치한다.

문제 22

$\text{gcd}(a,b)$ 는 두 정수 a,b 의 최대공약수를 반환하는 함수이다. a 는 0 이상의 정수이다. 두 값이 모두 0인 경우에만 $\text{gcd}(0,0)=0$ 으로 약속한다. $\text{gcd}(a,0)$ 의 반환값은?

- 1 $a-1$
- 2 1만 가능
- 3 정의할 수 없다
- 4 0만 가능
- 5 a

문제 22 정답과 해설

$\text{gcd}(a,b)$ 는 두 정수 a,b 의 최대공약수를 반환하는 함수이다. a 는 0 이상의 정수이다. 두 값이 모두 0인 경우에만 $\text{gcd}(0,0)=0$ 으로 약속한다. $\text{gcd}(a,0)$ 의 반환값은?

5번 a

해설

$a > 0$ 이면 $\text{gcd}(a,0)=a$ 이다.

두 수가 모두 0인 경우에는 이 문제의 정의에 따라 0을 반환한다.

문제 23

$a \geq 0$, $b > 0$ 인 정수에 유클리드 갱신 $(a, b) \leftarrow (b, a \bmod b)$ 를 적용한다. mod는 나머지 연산이다. 매번 더 작은 0 이상의 정수가 되어 종료를 보장하는 값은?

- 1 두 번째 인수 b
 - 2 첫 번째 인수 a , 초기 순서와 무관하게 항상 감소
 - 3 두 수의 공약수 개수
 - 4 두 수의 최대공약수
 - 5 두 수의 차 $a - b$, 항상 음이 아닌 채 감소
-
-

문제 23 정답과 해설

$a \geq 0, b > 0$ 인 정수에 유클리드 갱신 $(a, b) \leftarrow (b, a \bmod b)$ 를 적용한다. mod는 나머지 연산이다. 매번 더 작은 0 이상의 정수가 되어 종료를 보장하는 값은?

1번 두 번째 인수 b

해설

새 두 번째 인수는 0 이상 b 미만이다.

문제 24

정수 C 와 0 이상의 정수 k 가 주어진다. C 의 n 제곱 C^n 은 C 를 n 번 곱한 값이며, $C^0=1$ 로 정한다. $n=2k+1$ 인 홀수일 때, C^k 를 이용하여 C^n 을 올바르게 나타낸 식은?

- 1 $(C^k)^2$
- 2 $C+(C^k)^2$
- 3 C^k+C^{k+1}
- 4 $2 \cdot C^k$
- 5 $C \cdot (C^k)^2$

문제 24 정답과 해설

정수 C 와 0 이상의 정수 k 가 주어진다. C 의 n 제곱 C^n 은 C 를 n 번 곱한 값이며, $C^0=1$ 로 정한다. $n=2k+1$ 인 홀수일 때, C^k 를 이용하여 C^n 을 올바르게 나타낸 식은?

5번 $C \cdot (C^k)^2$

해설

C^k 를 두 번 곱하면 C 를 $2k$ 번 곱한 값이다. 필요한 횟수는 $2k+1$ 번이므로 C 를 한 번 더 곱한다.

문제 25

아래 Python 함수 A와 B는 정수 C의 n제곱(C를 n번 곱한 값)을 구한다.
n은 2, 4, 8, ... 중 하나이고, //는 정수 몫이다.
같은 인수로 다시 호출해도 결과를 자동으로 저장하지 않는다.

```
def A(C, n):  
    if n == 1: return C  
    return A(C, n // 2) * A(C, n // 2)  
def B(C, n):  
    if n == 1: return C  
    h = B(C, n // 2)  
    return h * h
```

곱셈·비교·대입 각각을 상수 시간으로 셀 때, A와 B의 전체 시간 복잡도는 순서대로 무엇인가?

- | | | | |
|---|--------------------------------|---|-----------------------------------|
| 1 | 둘 다 $\Theta(\log n)$ | 4 | 각각 $\Theta(n)$, $\Theta(\log n)$ |
| 2 | 각각 $\Theta(n^2)$, $\Theta(1)$ | 5 | 각각 $\Theta(\log n)$, $\Theta(n)$ |
| 3 | 둘 다 $\Theta(n^2)$ | | |

문제 25 정답과 해설

아래 Python 함수 A와 B는 정수 C의 n제곱(C를 n번 곱한 값)을 구한다.

n은 2, 4, 8, ... 중 하나이고, //는 정수 몫이다.

같은 인수로 다시 호출해도 결과를 자동으로 저장하지 않는다.

```
def A(C, n):  
    if n == 1: return C  
    return A(C, n // 2) * A(C, n // 2)  
def B(C, n):  
    if n == 1: return C  
    h = B(C, n // 2)  
    return h * h
```

곱셈·비교·대입 각각을 상수 시간으로 셀 때, A와 B의 전체 시간 복잡도는 순서대로 무엇인가?

4번 각각 $\Theta(n)$, $\Theta(\log n)$

해설

A는 매번 절반 크기의 같은 계산을 두 번 한다. 호출 수가 $1+2+4+\dots+n=2n-1$ 이므로 $\Theta(n)$ 이다. B는 한 번 계산한 결과 h를 재사용한다. 인수는 $n, n/2, \dots, 1$ 로 줄어 호출 수가 $\log_2 n + 1$ 이고 각 호출의 작업이 일정하므로 $\Theta(\log n)$ 이다.

문제 26

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

처음 $\text{power}(C, n)$ 을 호출할 때 $n \geq 2$ 이다. 최초 호출과 $n=0$ 인 마지막 호출을 모두 포함한 총 함수 호출 횟수는 n 에 대해 어떤 시간 차수로 증가하는가?

1 $\Theta(n)$

2 $\Theta(\log n)$

3 $\Theta(n \log n)$

4 $\Theta(n^2)$

5 $\Theta(1)$

문제 26 정답과 해설

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

처음 $\text{power}(C, n)$ 을 호출할 때 $n \geq 2$ 이다. 최초 호출과 $n=0$ 인 마지막 호출을 모두 포함한 총 함수 호출 횟수는 n 에 대해 어떤 시간 차수로 증가하는가?

2번 $\Theta(\log n)$

해설

매번 n 을 절반으로 줄이므로 0에 도달할 때까지의 총 호출 횟수는 $\Theta(\log n)$ 이다.

문제 27

길이 n 인 배열 a 를 오름차순 삽입 정렬한다. 인덱스는 0부터이다. $i=1$ 부터 차례로 현재 $a[i]$ 를 앞의 정렬된 구간에 삽입한다. 각 반복 i 가 시작되기 직전에 항상 성립하는 설명은? $a[p:q]$ 는 인덱스 p 부터 $q-1$ 까지의 구간이다.

- 1 $a[i:n]$ 가 정렬되어 있다
- 2 $a[i]$ 가 전체 최솟값이다
- 3 $a[0:i]$ 가 정렬되어 있다
- 4 앞부분 원소가 모두 같은 값이다
- 5 전체 배열이 정렬되어 있다

문제 27 정답과 해설

길이 n 인 배열 a 를 오름차순 삽입 정렬한다. 인덱스는 0부터이다. $i=1$ 부터 차례로 현재 $a[i]$ 를 앞의 정렬된 구간에 삽입한다. 각 반복 i 가 시작되기 직전에 항상 성립하는 설명은? $a[p:q]$ 는 인덱스 p 부터 $q-1$ 까지의 구간이다.

3번 $a[0:i]$ 가 정렬되어 있다

해설

정렬된 접두 구간에 현재 원소를 삽입하며 길이를 한 칸 늘린다.

문제 29

서로 다른 n 개 원소가 이미 오름차순으로 정렬되어 있다. 삽입 정렬은 두 번째 원소부터 하나씩 꺼내 앞부분을 뒤에서부터 검사하고, 꺼낸 값보다 큰 원소만 오른쪽으로 민다. 비교·대입을 상수 시간으로 셀 때 이 입력의 가장 정확한 시간 복잡도는?

- 1 $\Theta(n \log n)$
- 2 $\Theta(n^2)$ 가 반드시 필요하다
- 3 $\Theta(\log n)$
- 4 $\Theta(n)$
- 5 $\Theta(1)$

문제 29 정답과 해설

서로 다른 n 개 원소가 이미 오름차순으로 정렬되어 있다. 삽입 정렬은 두 번째 원소부터 하나씩 꺼내 앞부분을 뒤에서부터 검사하고, 꺼낸 값보다 큰 원소만 오른쪽으로 민다. 비교·대입을 상수 시간으로 셀 때 이 입력의 가장 정확한 시간 복잡도는?

4번 $\Theta(n)$

해설

각 삽입에서 한 번 비교하고 이동하지 않는다. 바깥 반복은 $n-1$ 번이다.

문제 30

서로 다른 n 개 원소가 내림차순인 배열을 오름차순 삽입 정렬한다. 앞 원소를 오른쪽으로 한 칸 미는 대입 $a[j+1]=a[j]$ 는 총 몇 회인가? key 저장과 마지막 삽입 대입은 제외한다.

1 $n-1$

2 $\lceil \log_2 n \rceil$

3 $n(n-1)/2$

4 2^n

5 n

문제 30 정답과 해설

서로 다른 n 개 원소가 내림차순인 배열을 오름차순 삽입 정렬한다. 앞 원소를 오른쪽으로 한 칸 미는 대입 $a[j+1]=a[j]$ 는 총 몇 회인가? key 저장과 마지막 삽입 대입은 제외한다.

3번 $n(n-1)/2$

해설

i 번째 삽입이 i 개를 이동시키므로 1부터 $n-1$ 까지 합한다.

문제 31

동전 중 가짜는 정확히 하나이며 진짜보다 가볍다. 진짜 동전은 모두 같은 무게이다. 후보 중 같은 개수의 두 더미를 양팔저울의 양쪽에 올렸더니 무게가 같았다. 가짜가 있는 곳에 대한 옳은 설명은?

- 1 가짜는 왼쪽 더미에 있다
- 2 가짜는 저울에 올리지 않은 후보들 중에 있다
- 3 무게 정보로 아무것도 줄일 수 없다
- 4 가짜가 두 개다
- 5 가짜는 오른쪽 더미에 있다

문제 31 정답과 해설

동전 중 가짜는 정확히 하나이며 진짜보다 가볍다. 진짜 동전은 모두 같은 무게이다. 후보 중 같은 개수의 두 더미를 양팔저울의 양쪽에 올렸더니 무게가 같았다. 가짜가 있는 곳에 대한 옳은 설명은?

2번 가짜는 저울에 올리지 않은 후보들 중에 있다

해설

저울 위에 가짜가 있었다면 두 더미의 무게가 달라진다.

문제 32

동전 27개 중 정확히 하나만 가볍고 진짜 동전의 무게는 모두 같다. 매번 후보를 같은 수의 세 그룹으로 나누고 두 그룹을 양팔저울로 비교해 후보 한 그룹만 남긴다. 후보가 1개가 될 때까지 최악의 저울질 횟수는?

1 3

2 4

3 2

4 9

5 27

문제 32 정답과 해설

동전 27개 중 정확히 하나만 가볍고 진짜 동전의 무게는 모두 같다. 매번 후보를 같은 수의 세 그룹으로 나누고 두 그룹을 양팔저울로 비교해 후보 한 그룹만 남긴다. 후보가 1개가 될 때까지 최악의 저울질 횟수는?

1번 3

해설

27→9→3→1로 후보를 줄인다. 각 비교의 세 결과를 활용한다.

문제 33

n 개 동전 중 하나만 가볍고 나머지 동전의 무게는 같다. n 은 2의 거듭제곱이다.
매번 후보를 같은 크기의 두 더미로 나누어 비교하고 가벼운 더미만 남긴다.
실물 양팔저울로 비교하는 횟수는 $O(\log n)$ 이다. 반면 프로그램에서 각 더미의 무게를 배열
원소를 하나씩 더해 구하면 전체 합산 시간은 $O(n)$ 이다.
두 결과가 동시에 맞을 수 있는 이유는?

- 1 재귀 깊이가 로그이면 합산 작업을 포함한 시간도 로그다
- 2 배열 합은 크기와 무관하게 항상 상수 시간이다
- 3 실물 저울질도 후보 수만큼 반복하므로 선형이다
- 4 두 분석 모두 배열의 각 원소를 상수 번만 더한다는 뜻이다
- 5 세는 기본 연산이 다르다

문제 33 정답과 해설

n 개 동전 중 하나만 가볍고 나머지 동전의 무게는 같다. n 은 2의 거듭제곱이다.

매번 후보를 같은 크기의 두 더미로 나누어 비교하고 가벼운 더미만 남긴다.

실물 양팔저울로 비교하는 횟수는 $O(\log n)$ 이다. 반면 프로그램에서 각 더미의 무게를 배열 원소를 하나씩 더해 구하면 전체 합산 시간은 $O(n)$ 이다.

두 결과가 동시에 맞을 수 있는 이유는?

5번 세는 기본 연산이 다르다

해설

실물 저울의 한 번 비교와 배열 원소를 더하는 작업의 비용 모형을 구분해야 한다.

문제 34

퀵 셀렉트는 n 개 원소 중 k 번째로 작은 값 하나를 찾는다. 퀵 정렬은 모든 원소를 순서대로 정렬한다. 두 방법 모두 기준값(피벗)과 각 원소를 비교해 그룹을 나눈다. 퀵 셀렉트가 퀵 정렬과 달리 보통 수행하지 않는 작업은?

- 1 원소와 피벗 비교하기
- 2 피벗 기준으로 구분하기
- 3 정답이 없는 쪽을 재귀적으로 정렬하기
- 4 순위에 따라 다음 구간 정하기
- 5 더 호출하지 않고 끝낼 조건을 확인하기

문제 34 정답과 해설

퀵 셀렉트는 n 개 원소 중 k 번째로 작은 값 하나를 찾는다. 퀵 정렬은 모든 원소를 순서대로 정렬한다. 두 방법 모두 기준값(피벗)과 각 원소를 비교해 그룹을 나눈다. 퀵 셀렉트가 퀵 정렬과 달리 보통 수행하지 않는 작업은?

3번 정답이 없는 쪽을 재귀적으로 정렬하기

해설

선택 문제는 한 순위의 값만 필요하므로 한쪽만 계속 처리한다.

문제 35

퀵 셀렉트에서 L 은 피벗보다 작은 원소 수, E 는 피벗과 같은 원소 수이다. 중복도 각각 세어 1부터 시작하는 순위 k 의 값을 찾을 때, 피벗이 정답인 조건은?

1 $k \geq L + E + 1$

2 $k > L + E$

3 $L < k \leq L + E$

4 $k = 0$

5 $k < L$

문제 35 정답과 해설

퀵 셀렉트에서 L 은 피벗보다 작은 원소 수, E 는 피벗과 같은 원소 수이다. 중복도 각각 세어 1부터 시작하는 순위 k 의 값을 찾을 때, 피벗이 정답인 조건은?

3번 $L < k \leq L + E$

해설

같은 값들이 정렬 결과의 $L+1$ 부터 $L+E$ 번째 순위를 차지한다.

문제 36

퀵 셀렉트에서 L 은 피벗보다 작은 원소 수, E 는 같은 원소 수이며 k 는 1부터 세는 순위이다. $k > L + E$ 라서 큰 값 구간으로 이동한다. 그 구간 안에서의 새 순위는?

- 1 $k - E$
 - 2 $k - L - E$
 - 3 $L + E - k$
 - 4 $k - L$
 - 5 $k + L + E$
-
-

문제 36 정답과 해설

퀵 선택에서 L 은 피벗보다 작은 원소 수, E 는 같은 원소 수이며 k 는 1부터 세는 순위이다. $k > L + E$ 라서 큰 값 구간으로 이동한다. 그 구간 안에서의 새 순위는?

2번 $k - L - E$

해설

버린 작은 값과 같은 값의 개수를 모두 뺀다.

문제 37

n 개 원소에서 k 번째로 작은 값을 퀵 셀렉트로 찾는다. 현재 후보가 m 개이면 한 번 분류하는데 $\Theta(m)$ 시간이 든다. 매번 정답 쪽에 $m-1$ 개가 남아, 후보 수가 $n, n-1, \dots, 1$ 로 줄어드는 경우 전체 시간 복잡도는?

- 1 $\Theta(\log n)$
- 2 $\Theta(n)$
- 3 $\Theta(n^2)$
- 4 $\Theta(2^n)$
- 5 $\Theta(n \log n)$

문제 37 정답과 해설

n 개 원소에서 k 번째로 작은 값을 퀵 셀렉트로 찾는다. 현재 후보가 m 개이면 한 번 분류하는 데 $\Theta(m)$ 시간이 든다. 매번 정답 쪽에 $m-1$ 개가 남아, 후보 수가 $n, n-1, \dots, 1$ 로 줄어드는 경우 전체 시간 복잡도는?

3번 $\Theta(n^2)$

해설

분류 비용은 $n + (n-1) + \dots + 2$ 에 비례한다. 이 합은 $\Theta(n^2)$ 이며, 후보가 한 개가 되면 반환한다.

문제 38

서로 다른 n 개 원소에서 k 번째로 작은 값을 퀵 셀렉트로 찾는다. 각 단계의 기준값(피벗)은 현재 후보 중에서 모두 같은 확률로 고른다. 무작위 선택 결과들을 확률에 따라 평균 낸 시간(기대 시간)과 가장 오래 걸리는 경우(최악 시간)의 옳은 설명은?

- 1 기대 선형 시간이나 최악 이차 시간이다
- 2 기대 시간은 이차이고 최악 시간은 지수다
- 3 기대 시간은 로그이고 최악 시간은 선형이다
- 4 기대 시간과 최악 시간이 모두 $n \log n$ 이다
- 5 기대 시간과 최악 시간이 모두 선형이다

문제 38 정답과 해설

서로 다른 n 개 원소에서 k 번째로 작은 값을 퀵 셀렉트로 찾는다. 각 단계의 기준값(피벗)은 현재 후보 중에서 모두 같은 확률로 고른다. 무작위 선택 결과들을 확률에 따라 평균 낸 시간(기대 시간)과 가장 오래 걸리는 경우(최악 시간)의 옳은 설명은?

1번 기대 선형 시간이나 최악 이차 시간이다

해설

확률적 기대 성능과 모든 실행에 대한 최악 보장은 다르다.

문제 39

퀵 셀렉트로 k번째로 작은 값을 찾는다. 원소를 기준값(피벗)보다 작은 값, 같은 값, 큰 값의 세 그룹으로 나누고, 중복된 값도 각각 순위에 센다. 같은 값이 많은 배열에서 이렇게 세 그룹으로 나누는 주된 이점은?

- 1 무작위 피벗 없이도 모든 입력의 최악 선형 시간을 보장한다
- 2 서로 다른 값에 대해서도 매번 정확히 반으로 나눈다
- 3 서로 다른 모든 원소까지 단 한 번의 비교로 정렬한다
- 4 같은 값 구간의 순위들을 한 번에 처리한다
- 5 파티션 단계의 원소 비교 수를 항상 상수로 만든다

문제 39 정답과 해설

퀵 셀렉트로 k 번째로 작은 값을 찾는다. 원소를 기준값(피벗)보다 작은 값, 같은 값, 큰 값의 세 그룹으로 나누고, 중복된 값도 각각 순위에 센다. 같은 값이 많은 배열에서 이렇게 세 그룹으로 나누는 주된 이점은?

4번 같은 값 구간의 순위들을 한 번에 처리한다

해설

같은 값 구간에 k 가 있으면 바로 답하고, 아니면 그 구간 전체를 제외한다.

문제 40

길이 n 인 배열에서 매번 현재 배열의 절반만 새 배열로 복사하여 다음 단계로 넘긴다. n 은 2의 거듭제곱이며 길이 1에서 끝낸다. 원소 한 개의 복사 비용은 일정하다. 따라서 현재 크기가 m 이면 복사 비용은 $\Theta(m)$ 이다. 단계 수만이 아니라 모든 단계의 복사 시간을 더한 전체 시간 복잡도는?

- 1 $\Theta(n)$
- 2 $\Theta(n \log n)$
- 3 $\Theta(\log n)$
- 4 $\Theta(1)$
- 5 $\Theta(n^2)$

문제 40 정답과 해설

길이 n 인 배열에서 매번 현재 배열의 절반만 새 배열로 복사하여 다음 단계로 넘긴다. n 은 2의 거듭제곱이며 길이 1에서 끝낸다. 원소 한 개의 복사 비용은 일정하다. 따라서 현재 크기가 m 이면 복사 비용은 $\Theta(m)$ 이다. 단계 수만이 아니라 모든 단계의 복사 시간을 더한 전체 시간 복잡도는?

1번 $\Theta(n)$

해설

복사되는 원소 수는 $n/2 + n/4 + \dots + 1 = n - 1$ 이다. 단계는 약 $\log_2 n$ 번이지만 각 단계의 복사량이 달라 전체 복사 시간은 $\Theta(n)$ 이다.

문제 41

다음 Python 함수는 0 이상의 정수 n 을 받아 팩토리얼 $n!$ 을 구한다. $0!=1$ 이며 $n \geq 1$ 이면 $n!=n \times (n-1) \times \dots \times 1$ 이다.

```
def fact(n):  
    if n == 0:  
        return 1  
    return n * fact(n - 1)
```

fact(5)의 ① 반환값 ② 총 함수 호출 횟수를 쓰시오. 최초 fact(5)와 마지막 fact(0) 호출을 모두 포함한다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 41 정답과 해설

다음 Python 함수는 0 이상의 정수 n 을 받아 팩토리얼 $n!$ 을 구한다. $0!=1$ 이며 $n \geq 1$ 이면 $n!=n \times (n-1) \times \dots \times 1$ 이다.

```
def fact(n):  
    if n == 0:  
        return 1  
    return n * fact(n - 1)
```

fact(5)의 ① 반환값 ② 총 함수 호출 횟수를 쓰시오. 최초 fact(5)와 마지막 fact(0) 호출을 모두 포함한다.

120, 6회

해설

인수는 5, 4, 3, 2, 1, 0이다. 곱셈은 5번이고 $n=0$ 인 호출까지 총 6번이다.

문제 42

다음 Python 함수를 fact(4)로 처음 실행한다.

```
def fact(n):  
    if n == 0:  
        return 1  
    return n * fact(n - 1)
```

- ① fact(0)이 실행되기 시작한 순간, 아직 반환하지 않은 함수 호출은 모두 몇 개인가?
fact(4)와 fact(0)도 포함한다.
- ② 그때 fact(0)이 반환하는 값은 무엇인가?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 42 정답과 해설

다음 Python 함수를 fact(4)로 처음 실행한다.

```
def fact(n):  
    if n == 0:  
        return 1  
    return n * fact(n - 1)
```

- ① fact(0)이 실행되기 시작한 순간, 아직 반환하지 않은 함수 호출은 모두 몇 개인가? fact(4)와 fact(0)도 포함한다.
- ② 그때 fact(0)이 반환하는 값은 무엇인가?

5개, 1

해설

fact(4), fact(3), fact(2), fact(1), fact(0)이 쌓이고 fact(0)이 1을 반환한다.

문제 43

Python에서 $a=0, b=1$ 로 시작해 다중 대입 $a, b=b, a+b$ 를 6번 수행한다. 오른쪽은 갱신 전 값으로 함께 계산한다. 최종 (a, b) 를 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 43 정답과 해설

Python에서 $a=0, b=1$ 로 시작해 다중 대입 $a, b=b, a+b$ 를 6번 수행한다. 오른쪽은 갱신 전 값으로 함께 계산한다. 최종 (a, b) 를 쓰시오.

(8,13)

해설

각 반복 후 (a, b) 는 연속된 피보나치 수이다. 6번 뒤 $F(6), F(7)$ 이다.

문제 44

다음 Python 함수에서 n 은 0 이상의 정수이다. $\text{fib}(0)=0$, $\text{fib}(1)=1$ 이며 이후 항은 앞의 두 항을 더한다. 이전 호출 결과는 저장하지 않는다.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

$\text{fib}(4)$ 를 한 번 실행하는 전체 과정에서 $\text{fib}(2)$ 는 몇 번 호출되는가?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 44 정답과 해설

다음 Python 함수에서 n 은 0 이상의 정수이다. $\text{fib}(0)=0$, $\text{fib}(1)=1$ 이며 이후 항은 앞의 두 항을 더한다. 이전 호출 결과는 저장하지 않는다.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

$\text{fib}(4)$ 를 한 번 실행하는 전체 과정에서 $\text{fib}(2)$ 는 몇 번 호출되는가?

2회

해설

$\text{fib}(4)$ 가 직접 $\text{fib}(2)$ 를 한 번 호출한다. 또 $\text{fib}(4)$ 가 호출한 $\text{fib}(3)$ 안에서 $\text{fib}(2)$ 를 한 번 더 호출하므로 총 2회이다.

문제 45

다음 Python 함수에서 n 은 0 이상의 정수이다. 이전 호출 결과는 저장하지 않는다.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

fib(4)를 한 번 실행할 때 함수 호출은 총 몇 회인가? 최초 fib(4)와 값을 바로 반환하는 fib(0), fib(1)도 한 번 호출될 때마다 각각 센다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 45 정답과 해설

다음 Python 함수에서 n 은 0 이상의 정수이다. 이전 호출 결과는 저장하지 않는다.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n - 1) + fib(n - 2)
```

fib(4)를 한 번 실행할 때 함수 호출은 총 몇 회인가? 최초 fib(4)와 값을 바로 반환하는 fib(0), fib(1)도 한 번 호출될 때마다 각각 센다.

9회

해설

호출된 인수별로 세면 fib(4) 1회, fib(3) 1회, fib(2) 2회, fib(1) 3회, fib(0) 2회이다. 합은 $1+1+2+3+2=9$ 회이다.

문제 46

배열 $[8, 3, 8, 2, 9]$ 를 왼쪽부터 순차 탐색해 첫 8의 인덱스를 반환한다. 인덱스는 0부터이다. 반환값과 배열 원소를 8과 비교한 횟수는? 반복 조건 검사는 제외한다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 46 정답과 해설

배열 [8,3,8,2,9]를 왼쪽부터 순차 탐색해 첫 8의 인덱스를 반환한다. 인덱스는 0부터이다. 반환값과 배열 원소를 8과 비교한 횟수는? 반복 조건 검사는 제외한다.

0, 1회

해설

첫 원소에서 즉시 일치하므로 뒤의 8은 확인하지 않는다.

문제 47

배열 $[8, 3, 8, 2, 9]$ 에서 7을 왼쪽부터 순차 탐색한다. 실패 시 -1 을 반환한다. 반환값과 배열 원소를 7과 비교한 횟수는? 반복 조건 검사는 제외한다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 47 정답과 해설

배열 $[8, 3, 8, 2, 9]$ 에서 7을 왼쪽부터 순차 탐색한다. 실패 시 -1 을 반환한다. 반환값과 배열 원소를 7과 비교한 횟수는? 반복 조건 검사는 제외한다.

$-1, 5$ 회

해설

모든 원소와 비교한 뒤 실패한다.

문제 48

배열 $a=[2,5,8,11,14,17,20]$ 에서 17을 이진 탐색한다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 인덱스는 0부터이며 초기 $(lo, hi)=(0, 6)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid=(lo+hi)//2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid] < 17$ 이면 $lo=mid+1$, $a[mid] > 17$ 이면 $hi=mid-1$ 로 바꾸고, 같으면 즉시 끝낸다. 확인한 mid 를 순서대로 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 48 정답과 해설

배열 $a=[2,5,8,11,14,17,20]$ 에서 17을 이진 탐색한다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 인덱스는 0부터이며 초기 $(lo, hi)=(0, 6)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid=(lo+hi)//2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid] < 17$ 이면 $lo=mid+1$, $a[mid] > 17$ 이면 $hi=mid-1$ 로 바꾸고, 같으면 즉시 끝낸다. 확인한 mid 를 순서대로 쓰시오.

3,5

해설

$mid=3$ 의 11보다 목표가 커서 $[4, 6]$ 으로 줄고 $mid=5$ 에서 찾는다.

문제 49

배열 $a=[2,5,8,11,14,17,20]$ 에서 6을 이진 탐색한다. 인덱스는 0부터이며 lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 처음에는 $(lo, hi)=(0, 6)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid=(lo+hi)//2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid]<6$ 이면 $lo=mid+1$, $a[mid]>6$ 이면 $hi=mid-1$ 이고, 같으면 즉시 끝낸다. 확인한 mid 를 순서대로 쓰고 종료 시 (lo, hi) 를 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 49 정답과 해설

배열 $a=[2,5,8,11,14,17,20]$ 에서 6을 이진 탐색한다. 인덱스는 0부터이며 lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 처음에는 $(lo, hi)=(0, 6)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid=(lo+hi)//2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid] < 6$ 이면 $lo=mid+1$, $a[mid] > 6$ 이면 $hi=mid-1$ 이고, 같으면 즉시 끝낸다. 확인한 mid 를 순서대로 쓰고 종료 시 (lo, hi) 를 쓰시오.

3,1,2; (2,1)

해설

11보다 작아 $[0, 2]$, 5보다 커서 $[2, 2]$, 8보다 작아 $hi=1$ 이 된다.

문제 50

오름차순 배열의 후보 인덱스가 양 끝을 포함한 $[0,9]$ 이다. 가운데 인덱스는 $(0+9)/2$ 를 내림해 고른다. 그 원소보다 목표값이 크면, 검사한 원소를 제외한 다음 후보 구간과 원소 수는?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 50 정답과 해설

오름차순 배열의 후보 인덱스가 양 끝을 포함한 $[0,9]$ 이다. 가운데 인덱스는 $(0+9)/2$ 를 내림해 고른다. 그 원소보다 목표값이 크면, 검사한 원소를 제외한 다음 후보 구간과 원소 수는?

$[5,9]$, 5개

해설

$mid=4$ 이며 mid 를 제외한 오른쪽 다섯 원소만 남는다.

문제 53

서로 다른 15개 원소가 오름차순인 배열 a 에서 목표값 x 를 이진 탐색한다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 인덱스는 0부터이며 초기 $(lo, hi) = (0, 14)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid = (lo + hi) // 2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid] = x$ 이면 즉시 끝낸다. 작으면 $lo = mid + 1$, 크면 $hi = mid - 1$ 로 바꾸고, $lo > hi$ 이면 실패로 끝낸다. 목표값이 존재하거나 존재하지 않는 모든 경우 중, 가운데 원소를 확인하는 반복 횟수의 최댓값은?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 53 정답과 해설

서로 다른 15개 원소가 오름차순인 배열 a 에서 목표값 x 를 이진 탐색한다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 인덱스는 0부터이며 초기 $(lo, hi) = (0, 14)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid = (lo + hi) // 2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid] = x$ 이면 즉시 끝낸다. 작으면 $lo = mid + 1$, 크면 $hi = mid - 1$ 로 바꾸고, $lo > hi$ 이면 실패로 끝낸다. 목표값이 존재하거나 존재하지 않는 모든 경우 중, 가운데 원소를 확인하는 반복 횟수의 최댓값은?

4회

해설

후보 수 $15 \rightarrow 7 \rightarrow 3 \rightarrow 1$ 을 차례로 검사한다. 루프 한 번의 목표값 비교 묶음을 한 회로 센다.

문제 54

오름차순 배열 $a = [10, 20, 30, 40, 50]$ 에서 값 40을 찾으려고 한다.

인덱스는 0부터 시작하며, $a[i]$ 는 인덱스 i 에 있는 값이다.

탐색 구간의 시작 인덱스 $lo = 0$, 끝 인덱스 $hi = 4$, 찾을 값 $x = 40$ 이다.

보간 탐색에서 처음 확인할 인덱스 pos 를 다음 식으로 계산한다.

$$pos = lo + ((x - a[lo]) * (hi - lo)) // (a[hi] - a[lo])$$

여기서 $*$ 는 곱셈, $//$ 는 나눗셈의 정수 몫이다. 예: $7 // 2 = 3$.

위 식에 주어진 값을 대입하여 인덱스 pos 를 구하시오.

인덱스 값과, 값을 대입한 계산식 한 줄을 함께 쓰시오.

문제 54 정답과 해설

$a = [10, 20, 30, 40, 50]$, $lo = 0$, $hi = 4$, $x = 40$

보간 탐색의 첫 확인 위치 pos 를 계산한다.

$pos = 3$ (네 번째 원소의 값은 40)

해설

양 끝의 값: $a[lo] = 10$, $a[hi] = 50$

$$\begin{aligned} pos &= 0 + ((40 - 10) \times (4 - 0)) // (50 - 10) \\ &= 120 // 40 = 3 \end{aligned}$$

인덱스는 0부터 세므로, 인덱스 3은 네 번째 원소이다.

문제 55

최대공약수를 구하는 유클리드 알고리즘은 $b \neq 0$ 인 동안 (a, b) 를 $(b, a \% b)$ 로 동시에 갱신한다. $\%$ 는 나머지 연산이다. $(84, 30)$ 에서 시작해 각 갱신 직후의 쌍을 종료할 때까지 순서대로 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 55 정답과 해설

최대공약수를 구하는 유클리드 알고리즘은 $b \neq 0$ 인 동안 (a, b) 를 $(b, a \% b)$ 로 동시에 갱신한다. %는 나머지 연산이다. $(84, 30)$ 에서 시작해 각 갱신 직후의 쌍을 종료할 때까지 순서대로 쓰시오.

$(30, 24) \rightarrow (24, 6) \rightarrow (6, 0)$

해설

$84 \bmod 30 = 24$, $30 \bmod 24 = 6$, $24 \bmod 6 = 0$ 이다. 최대공약수는 6이다.

문제 56

유클리드 알고리즘은 $b \neq 0$ 인 동안 (a, b) 를 $(b, a \% b)$ 로 동시에 바꾸고, $b = 0$ 이면 a 를 반환한다. $\%$ 는 나머지 연산이다. $(35, 64)$ 에서 시작할 때 첫 갱신 직후의 쌍과 최종 최대공약수를 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 56 정답과 해설

유클리드 알고리즘은 $b \neq 0$ 인 동안 (a, b) 를 $(b, a \% b)$ 로 동시에 바꾸고, $b = 0$ 이면 a 를 반환한다. %는 나머지 연산이다. $(35, 64)$ 에서 시작할 때 첫 갱신 직후의 쌍과 최종 최대공약수를 쓰시오.

$(64, 35), 1$

해설

$a < b$ 여도 $a \bmod b = a$ 이므로 먼저 자리가 바뀐다. 이후 $64, 35, 29, 6, 5, 1, 0$ 으로 진행한다.

문제 57

유클리드 알고리즘은 $b \neq 0$ 인 동안 (a, b) 를 $(b, a \% b)$ 로 바꾸고 $b = 0$ 이면 a 를 반환한다. $\%$ 는 나머지 연산이다. $\text{gcd}(0, 18)$ 을 이 규칙 그대로 실행할 때 $\%$ 연산 횟수와 반환값은?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 57 정답과 해설

유클리드 알고리즘은 $b \neq 0$ 인 동안 (a, b) 를 $(b, a \% b)$ 로 바꾸고 $b = 0$ 이면 a 를 반환한다. $\%$ 는 나머지 연산이다. $\text{gcd}(0, 18)$ 을 이 규칙 그대로 실행할 때 $\%$ 연산 횟수와 반환값은?

1회, 18

해설

한 번 갱신하면 $(18, 0)$ 이므로 종료한다.

문제 58

`power(C,n)`은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 `//`는 정수 몫, `%`는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

`power(2,13)`을 실행한다. 최초 호출부터 $n=0$ 인 마지막 호출까지, 함수에 전달되는 n 의 값을 순서대로 쓰시오. 반환값이나 곱셈 횟수는 구하지 않는다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 58 정답과 해설

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

$\text{power}(2, 13)$ 을 실행한다. 최초 호출부터 $n=0$ 인 마지막 호출까지, 함수에 전달되는 n 의 값을 순서대로 쓰시오. 반환값이나 곱셈 횟수는 구하지 않는다.

13 → 6 → 3 → 1 → 0

해설

다음 호출의 두 번째 인수는 $n//2$ 이다. $13//2=6$, $6//2=3$, $3//2=1$, $1//2=0$ 이며 $n=0$ 에서는 더 호출하지 않는다.

문제 59

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

$\text{power}(2, 13)$ 을 실행할 때 * 연산은 총 몇 회인가? 모든 재귀 호출에서 실행한 *를 합산한다. 1을 곱해도 * 한 번은 1회이고, $n=0$ 에서 1을 반환할 때는 곱셈하지 않는다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 59 정답과 해설

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

$\text{power}(2, 13)$ 을 실행할 때 * 연산은 총 몇 회인가? 모든 재귀 호출에서 실행한 *를 합산한다. 1을 곱해도 * 한 번은 1회이고, $n=0$ 에서 1을 반환할 때는 곱셈하지 않는다.

7회

해설

n 이 13, 6, 3, 1인 네 호출에서 $h * h$ 를 각 1회 실행한다. 그중 13, 3, 1은 홀수이므로 $r * C$ 를 각 1회 더 실행한다. $n=0$ 은 곱셈 0회이므로 전체는 $4+3=7$ 회이다.

문제 60

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

$\text{power}(3, 5)$ 의 ① 반환값과 ② 모든 호출에서 실행한 $*$ 연산의 총횟수를 쓰시오. 1을 곱해도 $*$ 한 번은 1회이고, $n=0$ 에서 1을 반환할 때는 곱셈하지 않는다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 60 정답과 해설

$\text{power}(C, n)$ 은 정수 C 의 n 제곱, 즉 C 를 n 번 곱한 값을 반환한다. n 은 0 이상의 정수이며 $C^0=1$ 로 정한다. Python에서 $//$ 는 정수 몫, $\%$ 는 나머지이다.

```
def power(C, n):  
    if n == 0:  
        return 1  
    h = power(C, n // 2)  
    r = h * h  
    if n % 2 == 1: r = r * C  
    return r
```

$\text{power}(3, 5)$ 의 ① 반환값과 ② 모든 호출에서 실행한 $*$ 연산의 총횟수를 쓰시오. 1을 곱해도 $*$ 한 번은 1회이고, $n=0$ 에서 1을 반환할 때는 곱셈하지 않는다.

① 243 ② 5회

해설

호출되는 n 은 5, 2, 1, 0이다. $n=0$ 의 반환값은 1이며, 돌아오면서 $n=1$ 에서 $1 \times 1 \times 3 = 3$ (2회), $n=2$ 에서 $3 \times 3 = 9$ (1회), $n=5$ 에서 $9 \times 9 \times 3 = 243$ (2회)을 계산한다. 총 $2+1+2=5$ 회이다.

문제 61

인덱스가 0부터인 $[5, 2, 4, 1]$ 을 오름차순 삽입 정렬한다. $i=1$ 부터 시작해 현재 $a[i]$ 를 앞의 정렬된 구간 $a[0] \sim a[i-1]$ 에 삽입한다. $i=1$ 과 $i=2$ 의 삽입을 마친 직후 배열을 각각 쓰시오.

두 단계의 배열을 각각 쓰시오. 별도의 과정 설명은 필요하지 않습니다.

문제 61 정답과 해설

인덱스가 0부터인 $[5, 2, 4, 1]$ 을 오름차순 삽입 정렬한다. $i=1$ 부터 시작해 현재 $a[i]$ 를 앞의 정렬된 구간 $a[0] \sim a[i-1]$ 에 삽입한다. $i=1$ 과 $i=2$ 의 삽입을 마친 직후 배열을 각각 쓰시오.

$[2, 5, 4, 1], [2, 4, 5, 1]$

해설

2를 5 앞에, 다음 4를 2와 5 사이에 삽입한다.

문제 62

배열 $[4,3,2,1]$ 을 오름차순 삽입 정렬한다. 두 번째 원소부터 현재 값을 key 에 보관하고, 앞부분에서 key 보다 큰 원소를 오른쪽으로 한 칸씩 민 뒤 key 를 넣는다. 원소를 한 칸 미는 대입은 총 몇 회인가? key 를 보관하거나 마지막에 넣는 대입은 세지 않는다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 62 정답과 해설

배열 [4,3,2,1]을 오름차순 삽입 정렬한다. 두 번째 원소부터 현재 값을 key에 보관하고, 앞부분에서 key보다 큰 원소를 오른쪽으로 한 칸씩 민 뒤 key를 넣는다. 원소를 한 칸 미는 대입은 총 몇 회인가? key를 보관하거나 마지막에 넣는 대입은 세지 않는다.

6회

해설

각 삽입에서 1,2,3개를 이동시킨다.

문제 64

1번부터 9번까지 동전 중 정확히 하나가 가볍고, 진짜 동전의 무게는 모두 같다. 양팔저울에 1~3번과 4~6번을 올렸더니 무게가 같았다. ① 가짜일 수 있는 동전 번호를 쓰시오. ② 이 결과를 알고 난 뒤, 가짜를 반드시 찾는 데 추가로 필요한 저울질의 최소 횟수는 최악의 경우 몇 회인가?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 64 정답과 해설

1번부터 9번까지 동전 중 정확히 하나가 가볍고, 진짜 동전의 무게는 모두 같다. 양팔저울에 1~3번과 4~6번을 올렸더니 무게가 같았다. ① 가짜일 수 있는 동전 번호를 쓰시오. ② 이 결과를 알고 난 뒤, 가짜를 반드시 찾는 데 추가로 필요한 저울질의 최소 횟수는 최악의 경우 몇 회인가?

7~9번, 1회

해설

7번과 8번을 비교하면 가벼운 쪽 또는 균형일 때 9번을 고를 수 있다.

문제 65

동전 81개 중 정확히 하나만 가볍고 진짜 무게는 모두 같다. 매번 후보를 같은 수의 세 그룹으로 나누어 두 그룹을 비교하고 가짜가 있는 한 그룹만 남긴다. 두 번 저울질한 뒤 최대 후보 수는?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 65 정답과 해설

동전 81개 중 정확히 하나만 가볍고 진짜 무게는 모두 같다. 매번 후보를 같은 수의 세 그룹으로 나누어 두 그룹을 비교하고 가짜가 있는 한 그룹만 남긴다. 두 번 저울질한 뒤 최대 후보 수는?

9개

해설

81→27→9이다. 각 단계에서 같은 크기 세 그룹으로 나눈다.

문제 66

배열 $[7, 2, 5, 2, 9, 4]$ 를 기준값(피벗) 4보다 작은 값, 4와 같은 값, 4보다 큰 값의 세 그룹으로 나눈다. 각 그룹 안에서는 원래 배열에 나타난 순서를 유지한다. 세 그룹의 배열을 작은 값, 같은 값, 큰 값 순서로 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 66 정답과 해설

배열 [7,2,5,2,9,4]를 기준값(피벗) 4보다 작은 값, 4와 같은 값, 4보다 큰 값의 세 그룹으로 나눈다. 각 그룹 안에서는 원래 배열에 나타난 순서를 유지한다. 세 그룹의 배열을 작은 값, 같은 값, 큰 값 순서로 쓰시오.

[2,2], [4], [7,5,9]

해설

각 원소를 4와 비교하여 세 구간에 넣는다.

문제 67

배열 [7,2,5,2,9,4]에서 중복된 값도 각각 세어 5번째로 작은 값을 찾는다. 기준값 4보다 작은 값, 같은 값, 큰 값의 세 그룹으로 나눈다. 작은 그룹과 같은 그룹을 제외하고 큰 그룹에서 계속 찾는다. ① 큰 그룹 안에서는 몇 번째로 작은 값을 찾아야 하는가? ② 최종 찾는 값은 무엇인가? 순위는 1부터 센다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 67 정답과 해설

배열 [7,2,5,2,9,4]에서 중복된 값도 각각 세어 5번째로 작은 값을 찾는다. 기준값 4보다 작은 값, 같은 값, 큰 값의 세 그룹으로 나눈다. 작은 그룹과 같은 그룹을 제외하고 큰 그룹에서 계속 찾는다. ① 큰 그룹 안에서는 몇 번째로 작은 값을 찾아야 하는가? ② 최종 찾는 값은 무엇인가? 순위는 1부터 센다.

2번째, 7

해설

작은 값 2개와 같은 값 1개를 제외하면 새 순위는 $5 - 2 - 1 = 2$ 이다.
큰 값 [7,5,9] 중 두 번째로 작은 값은 7이다.

문제 68

[4,1,4,2,4,7]에서 중복도 각각 세어 4번째로 작은 값을 찾는다. 피벗 4로 분할할 때 L은 4보다 작은 원소 수, E는 4와 같은 원소 수이다. L,E와 찾는 값을 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 68 정답과 해설

[4,1,4,2,4,7]에서 중복도 각각 세어 4번째로 작은 값을 찾는다. 피벗 4로 분할할 때 L은 4보다 작은 원소 수, E는 4와 같은 원소 수이다. L,E와 찾는 값을 쓰시오.

L=2,E=3, 답=4

해설

같은 값 구간이 3~5번째 순위를 차지한다.

문제 69

$T(n)$ 은 입력 크기 n 에서 수행하는 전체 기본 연산 횟수이다. 크기 1에서는 연산을 1회 한다. 크기 $n > 1$ 에서는 연산 n 회를 한 다음 크기 $n/2$ 의 문제 하나를 푼다. n 은 1, 2, 4, 8, ... 중 하나이다. 따라서 $T(1)=1$, $T(n)=n+T(n/2)$ 이다. $T(8)$ 의 정확한 값을 구하시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 69 정답과 해설

$T(n)$ 은 입력 크기 n 에서 수행하는 전체 기본 연산 횟수이다. 크기 1에서는 연산을 1회 한다. 크기 $n > 1$ 에서는 연산 n 회를 한 다음 크기 $n/2$ 의 문제 하나를 푼다. n 은 1, 2, 4, 8, ... 중 하나이다. 따라서 $T(1)=1$, $T(n)=n+T(n/2)$ 이다. $T(8)$ 의 정확한 값을 구하시오.

15

해설

$T(8)=8+4+2+1=15$ 이다. 마지막 $T(1)=1$ 의 비용도 더한다.

문제 70

27행 27열 별무늬를 만드는 함수가 있다. 크기가 1이면 별 하나를 반환하고 더 호출하지 않는다. 그 외에는 크기를 3으로 나눈 무늬를 같은 함수로 한 번 구해 재사용한다. ① 크기 27의 최초 호출부터 크기 1의 마지막 호출까지 총 몇 번 호출하는가? ② 최종 27행 27열을 모두 출력할 때 공백을 포함한 문자 수는 몇 개인가? 줄바꿈은 세지 않는다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 70 정답과 해설

27행 27열 별무늬를 만드는 함수가 있다. 크기가 1이면 별 하나를 반환하고 더 호출하지 않는다. 그 외에는 크기를 3으로 나눈 무늬를 같은 함수로 한 번 구해 재사용한다. ① 크기 27의 최초 호출부터 크기 1의 마지막 호출까지 총 몇 번 호출하는가? ② 최종 27행 27열을 모두 출력할 때 공백을 포함한 문자 수는 몇 개인가? 줄바꿈은 세지 않는다.

① 4회 ② 729개

해설

함수에 전달되는 크기는 27, 9, 3, 1이므로 4회 호출한다. 최종 출력은 27행×27열=729개 위치이며, 각 위치의 별 또는 공백을 하나의 문자로 센다.

문제 71

다음 Python 코드에서 N 은 2,4,8,... 중 하나이다. $//$ 는 나눗셈의 정수 몫이다. $\text{count} += 1$ 한 번을 기본 연산 1회로 센다.

```
n = N
count = 0
while n > 0:
    for i in range(n):
        count += 1
    n = n // 2
```

- ① 각 while 반복에서 count가 증가하는 횟수를 N 을 사용해 순서대로 쓰시오.
- ② n 이 계속 절반으로 줄어도 전체 시간이 $O(\log N)$ 이 아닌 이유를 설명하고, 전체 시간 복잡도를 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 71 정답과 해설

다음 Python 코드에서 N 은 2,4,8,... 중 하나이다. $//$ 는 나눗셈의 정수 몫이다. $\text{count} += 1$ 한 번을 기본 연산 1회로 센다.

```
n = N
count = 0
while n > 0:
    for i in range(n):
        count += 1
    n = n // 2
```

- ① 각 while 반복에서 count 가 증가하는 횟수를 N 을 사용해 순서대로 쓰시오.
- ② n 이 계속 절반으로 줄어도 전체 시간이 $O(\log N)$ 이 아닌 이유를 설명하고, 전체 시간 복잡도를 쓰시오.

① $N, N/2, N/4, \dots, 1$ ② 전체 시간은 $\Theta(N)$ 이다.

해설

while 반복은 $\log_2 N + 1$ 번이지만 각 단계에서 for 반복 횟수가 다르다. count 증가 횟수는 $N + N/2 + \dots + 1 = 2N - 1$ 이다. 첫 단계만도 N 회이며 전체도 N 에 비례하므로 $\Theta(N)$ 이다. 따라서 절반 축소만 보고 전체 시간을 $O(\log N)$ 이라고 할 수 없다.

문제 72

아래 Python 코드는 오름차순 배열 a 에서 x 의 인덱스를 찾으려 한다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이고 mid 는 가운데 인덱스이다. 인덱스는 0부터이며 $//$ 는 정수 몫이다.

```
lo, hi = 0, len(a) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == x:
        break
    if a[mid] < x:
        lo = mid
    else:
        hi = mid - 1
```

$lo = mid$ 때문에 while 반복이 끝나지 않는 최소 길이의 배열 a 와 목표값 x 를 하나 제시하고, 반복해도 구간이 바뀌지 않는 이유를 설명하시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 72 정답과 해설

아래 Python 코드는 오름차순 배열 a에서 x의 인덱스를 찾으려 한다. lo,hi는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이고 mid는 가운데 인덱스이다. 인덱스는 0부터이며 //는 정수 몫이다.

```
lo, hi = 0, len(a) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == x:
        break
    if a[mid] < x:
        lo = mid
    else:
        hi = mid - 1
```

lo = mid 때문에 while 반복이 끝나지 않는 최소 길이의 배열 a와 목표값 x를 하나 제시하고, 반복해도 구간이 바뀌지 않는 이유를 설명하시오.

길이 1: a=[1], x=2이면 lo=hi=mid=0이 반복된다.

해설

1<2이므로 lo=mid=0으로 갱신해도 후보 구간이 그대로이다.
빈 배열은 반복을 시작하지 않으므로 최소 길이는 1이다.
올바른 갱신 lo=mid+1은 검사한 원소를 제외하고 종료시킨다.

문제 74

배열 $a=[4,1,3]$ 에서 4를 찾는다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 인덱스는 0부터이며 초기 $(lo, hi)=(0, 2)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid=(lo+hi)//2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid]=4$ 이면 성공으로 끝내고, 작으면 $lo=mid+1$, 크면 $hi=mid-1$ 로 바꾼다. $lo > hi$ 이면 실패로 끝낸다. 실제로 4가 있는데도 탐색이 실패하는 이유를, 처음 확인한 값과 버려지는 구간을 들어 설명하시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 74 정답과 해설

배열 $a=[4,1,3]$ 에서 4를 찾는다. lo, hi 는 양 끝을 포함한 후보 구간의 첫·마지막 인덱스이다. 인덱스는 0부터이며 초기 $(lo, hi)=(0, 2)$ 이다. $lo \leq hi$ 인 동안 가운데 인덱스 $mid=(lo+hi)//2$ 를 확인한다. $//$ 는 정수 몫이다. $a[mid]=4$ 이면 성공으로 끝내고, 작으면 $lo=mid+1$, 크면 $hi=mid-1$ 로 바꾼다. $lo > hi$ 이면 실패로 끝낸다. 실제로 4가 있는데도 탐색이 실패하는 이유를, 처음 확인한 값과 버려지는 구간을 들어 설명하시오.

mid=1의 값 1을 보고 왼쪽을 버리면 실제 4도 버린다.

해설

절반을 버리는 근거는 정렬 순서이다. 중간값 비교만으로 무정렬 구간의 나머지 값을 판단할 수 없다.

문제 75

배열 a 에는 원소가 하나 이상 있고 모든 값이 5이다. lo 와 hi 는 양 끝을 포함한 후보 구간의 시작·끝 인덱스이고, 찾는 값은 x 이다. 보간 탐색의 위치 추정식에는 $a[hi] - a[lo]$ 로 나누는 계산이 있다. ① 이 나눗셈을 그대로 실행하면 어떤 문제가 생기는가? ② $x=5$ 인 경우와 $x \neq 5$ 인 경우 각각, 탐색을 어떻게 끝내야 하는가?

풀이 과정과 판단 근거를 함께 쓰시오.

문제 75 정답과 해설

배열 a 에는 원소가 하나 이상 있고 모든 값이 5이다. lo 와 hi 는 양 끝을 포함한 후보 구간의 시작·끝 인덱스이고, 찾는 값은 x 이다. 보간 탐색의 위치 추정식에는 $a[hi] - a[lo]$ 로 나누는 계산이 있다. ① 이 나눗셈을 그대로 실행하면 어떤 문제가 생기는가? ② $x=5$ 인 경우와 $x \neq 5$ 인 경우 각각, 탐색을 어떻게 끝내야 하는가?

① 0으로 나누게 된다. ② $x=5$ 이면 성공, $x \neq 5$ 이면 실패로 끝낸다.

해설

양 끝 값이 모두 5이므로 분모는 $5 - 5 = 0$ 이다. 모든 원소가 5라고 주어졌으므로 $x=5$ 일 때는 어느 후보 위치도 일치한다. 다른 값을 찾을 때는 일치하는 원소가 없으므로 위치 식을 계산할 필요가 없다.

문제 76

n 개 번호($n \geq 2$)가 든 목록에서 번호를 찾는 요청이 m 번($m \geq 1$) 들어온다. 요청마다 찾는 번호는 달라도 되며 목록 자체는 바뀌지 않는다.

방법 A: 요청이 올 때마다 첫 원소부터 하나씩 순차 탐색한다.

방법 B: 처음 한 번 오름차순으로 정렬한 뒤, 매 요청을 이진 탐색한다.

정렬 한 번의 시간은 $\Theta(n \log n)$, 원소 한 번의 비교는 상수 시간으로 주어진다.

① 각 방법의 m 번 요청을 합친 최악의 총 시간 복잡도를 쓰시오. B는 처음 정렬하는 시간도 포함한다.

② 요청이 적을 때와 같은 목록을 여러 번 검색할 때 중, 어느 상황에서 B를 고려하는지 이유를 설명하시오. 정확한 손익분기 횟수는 구하지 않는다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 76 정답과 해설

n 개 번호($n \geq 2$)가 든 목록에서 번호를 찾는 요청이 m 번($m \geq 1$) 들어온다. 요청마다 찾는 번호는 달라도 되며 목록 자체는 바뀌지 않는다.

방법 A: 요청이 올 때마다 첫 원소부터 하나씩 순차 탐색한다.

방법 B: 처음 한 번 오름차순으로 정렬한 뒤, 매 요청을 이진 탐색한다.

정렬 한 번의 시간은 $\Theta(n \log n)$, 원소 한 번의 비교는 상수 시간으로 주어진다.

- ① 각 방법의 m 번 요청을 합친 최악의 총 시간 복잡도를 쓰시오. B는 처음 정렬하는 시간도 포함한다.
- ② 요청이 적을 때와 같은 목록을 여러 번 검색할 때 중, 어느 상황에서 B를 고려하는지 이유를 설명하시오. 정확한 손익분기 횟수는 구하지 않는다.

① A: $\Theta(mn)$, B: $\Theta(n \log n + m \log n)$

② 같은 목록을 여러 번 검색할 때 B를 고려한다.

해설

A는 요청마다 최악 n 개를 비교하므로 m 번이면 $\Theta(mn)$ 이다. B는 정렬 비용을 한 번만 들이고 요청당 최악 $\Theta(\log n)$ 을 쓴다. 검색이 적으면 초기 정렬 비용이 클 수 있지만 검색이 많아지면 요청당 탐색 비용을 줄이는 이점이 커진다.

문제 77

최대공약수를 구하는 유클리드 알고리즘은 b 가 0이 아닐 동안 (a,b) 를 $(b,a\%b)$ 로 동시에 바꾼다. $\%$ 는 a 를 b 로 나눈 나머지이며, $b=0$ 이면 a 를 반환한다. $(18,12)$ 에서 시작한다. 시작 쌍부터 종료할 때까지의 (a,b) 를 순서대로 쓰고, 최종 최대공약수를 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 77 정답과 해설

최대공약수를 구하는 유클리드 알고리즘은 b 가 0이 아닐 동안 (a,b) 를 $(b,a\%b)$ 로 동시에 바꾼다. $\%$ 는 a 를 b 로 나눈 나머지이며, $b=0$ 이면 a 를 반환한다. $(18,12)$ 에서 시작한다. 시작 쌍부터 종료할 때까지의 (a,b) 를 순서대로 쓰고, 최종 최대공약수를 쓰시오.

$(18,12) \rightarrow (12,6) \rightarrow (6,0)$, 최대공약수 6

해설

$18\%12=6$, $12\%6=0$ 이다. 두 번째 값이 0이 되었을 때 첫 번째 값 6이 최대공약수다.

문제 78

$\text{power}(C, k)$ 는 C 의 k 제곱을 계산해 반환하는 함수이다.

예를 들어 $\text{power}(2, 4)$ 는 $2 \times 2 \times 2 \times 2$ 인 16을 반환한다.

이 함수는 호출할 때마다 다시 계산하며, 이전 결과를 자동으로 저장하지 않는다.

$2^8 = 2^4 \times 2^4$ 를 계산하는 다음 두 코드를 비교하시오.

코드 A:

```
h = power(2, 4)
```

```
result = h * h
```

코드 B:

```
result = power(2, 4) * power(2, 4)
```

- ① 코드 A와 B에서 $\text{power}(2, 4)$ 를 각각 몇 번 호출하는가?
- ② 같은 거듭제곱을 두 번 계산하는 쪽을 고르고, 왜 계산이 더 필요한지 설명하시오.

① 두 호출 횟수 ② 중복 계산하는 코드와 그 이유를 쓰시오.

문제 78 정답과 해설

$\text{power}(C, k)$ 는 C 의 k 제곱을 계산해 반환하는 함수이다.

예를 들어 $\text{power}(2, 4)$ 는 $2 \times 2 \times 2 \times 2$ 인 16을 반환한다.

이 함수는 호출할 때마다 다시 계산하며, 이전 결과를 자동으로 저장하지 않는다.

$2^8 = 2^4 \times 2^4$ 를 계산하는 다음 두 코드를 비교하시오.

코드 A:

```
h = power(2, 4)
```

```
result = h * h
```

코드 B:

```
result = power(2, 4) * power(2, 4)
```

- ① 코드 A와 B에서 $\text{power}(2, 4)$ 를 각각 몇 번 호출하는가?
- ② 같은 거듭제곱을 두 번 계산하는 쪽을 고르고, 왜 계산이 더 필요한지 설명하시오.

① A는 1회, B는 2회. ② B가 같은 거듭제곱을 두 번 계산한다.

해설

A는 $\text{power}(2, 4)$ 의 계산 결과를 h 에 저장하고 h 를 두 번 사용한다. B는 동일한 함수를 두 번 호출하며, 호출할 때마다 2^4 를 다시 계산한다. 두 코드 모두 마지막 곱셈은 한 번이지만 B는 거듭제곱 계산을 한 번 더 수행한다. 정확한 시간 복잡도나 result 의 값은 이 문제에서 요구하지 않는다.

문제 79

다음 C 코드는 피보나치 수열의 다음 두 항을 계산하려는 코드이다.
갱신 전 a 는 현재 항, b 는 다음 항이다. 갱신 후에는 각각
기존 b 와 기존 $a + b$ 가 되어야 한다.

```
a = b;  
b = a + b;
```

- ① 갱신 전 $a = 2$, $b = 3$ 일 때, 위 코드 실행 후 a 와 b 의 값을 쓰시오.
- ② 의도한 결과와 달라지는 이유를 설명하고, 임시 변수를 사용하여 수정하시오.

배점: 실행 결과 2점 · 오류 원인 3점 · 수정안 5점. C 코드로 수정안을 쓰시오.

문제 79 정답과 해설

$a = 2, b = 3$ 에서 순차 대입의 결과를 확인하고 수정한다.

① 실제 결과: $a = 3, b = 6$ / 의도한 결과: $a = 3, b = 5$

해설

② $a = b;$ 를 실행하면 기존 a 의 값 2가 사라진다.
따라서 $b = a + b;$ 는 $2 + 3$ 이 아니라 $3 + 3$ 을 계산한다.

```
int t = a + b;  
a = b;  
b = t;
```

덮어쓰기 전에 합 5를 t 에 저장하므로, 갱신 후 $a = 3, b = 5$ 가 된다.

문제 81

삽입 정렬은 두 번째 원소부터 현재 값을 key에 보관하고, 앞부분에서 key보다 큰 원소를 오른쪽으로 한 칸씩 민 뒤 key를 넣는다. key는 이번에 삽입할 값이다.

서로 다른 n 개 원소($n \geq 2$)의 두 입력 $A=[1,2,\dots,n]$ 과 $B=[n,n-1,\dots,1]$ 을 이 방법으로 오름차순 정렬한다.

앞 원소를 한 칸 미는 대입만 '이동 1회'로 세며, key 저장과 마지막 삽입은 이동 수에서 제외한다.

- ① A와 B에서 전체 이동 횟수와 시간 복잡도는 각각 무엇인가?
- ② 둘 다 삽입 단계는 $n-1$ 번인데 전체 시간이 달라지는 이유를 설명하시오. 비교·대입은 상수 시간이다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 81 정답과 해설

삽입 정렬은 두 번째 원소부터 현재 값을 key에 보관하고, 앞부분에서 key보다 큰 원소를 오른쪽으로 한 칸씩 민 뒤 key를 넣는다. key는 이번에 삽입할 값이다.

서로 다른 n 개 원소($n \geq 2$)의 두 입력 $A=[1,2,\dots,n]$ 과 $B=[n,n-1,\dots,1]$ 을 이 방법으로 오름차순 정렬한다.

앞 원소를 한 칸 미는 대입만 '이동 1회'로 세며, key 저장과 마지막 삽입은 이동 수에서 제외한다.

- ① A와 B에서 전체 이동 횟수와 시간 복잡도는 각각 무엇인가?
- ② 둘 다 삽입 단계는 $n-1$ 번인데 전체 시간이 달라지는 이유를 설명하시오. 비교·대입은 상수 시간이다.

① A: 이동 0회, $\Theta(n)$. B: 이동 $n(n-1)/2$ 회, $\Theta(n^2)$.

② 각 삽입에서 앞 원소를 옮기는 횟수가 다르다.

해설

A는 각 단계에서 바로 앞 원소와 한 번 비교하면 이동 없이 끝난다. B는 1개, 2개, ..., $n-1$ 개를 차례로 이동하므로 총 $1+2+\dots+(n-1)=n(n-1)/2$ 회이다. 바깥 단계가 같아도 각 단계의 작업량을 더해야 전체 시간을 알 수 있다.

문제 82

진짜 동전은 모두 같은 무게이고 가짜는 정확히 하나이다. 기존 방법은 같은 개수의 두 더미를 양팔저울로 비교하여, 무게가 다르면 가벼운 더미만 후보로 남긴다. 가짜가 더 가벼운지 더 무거운지 모르는 상황에서 이 방법을 그대로 쓰면 안 되는 이유를 설명하시오. 새 동전 찾기 알고리즘을 설계할 필요는 없다.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 82 정답과 해설

진짜 동전은 모두 같은 무게이고 가짜는 정확히 하나이다. 기존 방법은 같은 개수의 두 더미를 양팔저울로 비교하여, 무게가 다르면 가벼운 더미만 후보로 남긴다. 가짜가 더 가벼운지 더 무거운지 모르는 상황에서 이 방법을 그대로 쓰면 안 되는 이유를 설명하시오. 새 동전 찾기 알고리즘을 설계할 필요는 없다.

가짜가 무거운 동전일 수도 있으므로 가벼운 더미만 남기면 실제 가짜를 버릴 수 있다.

해설

같은 개수인 두 더미의 무게가 다르면, 가벼운 더미에 가벼운 가짜가 있거나 무거운 더미에 무거운 가짜가 있을 수 있다. 어느 경우인지 주어지지 않아 한쪽을 확정할 수 없다.

문제 83

동전 27개 중 가짜는 정확히 하나이며 진짜보다 가볍고, 진짜 동전의 무게는 모두 같다. 매번 후보를 같은 수의 세 그룹으로 나누고 두 그룹을 양팔저울로 비교한다. 무게가 다르면 가벼운 그룹, 같으면 저울에 올리지 않은 그룹만 남긴다. 후보가 1개가 될 때까지의 후보 수 변화를 처음 27개부터 쓰고, 저울질 총횟수를 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 83 정답과 해설

동전 27개 중 가짜는 정확히 하나이며 진짜보다 가볍고, 진짜 동전의 무게는 모두 같다. 매번 후보를 같은 수의 세 그룹으로 나누고 두 그룹을 양팔저울로 비교한다. 무게가 다르면 가벼운 그룹, 같으면 저울에 올리지 않은 그룹만 남긴다. 후보가 1개가 될 때까지의 후보 수 변화를 처음 27개부터 쓰고, 저울질 총횟수를 쓰시오.

27 → 9 → 3 → 1, 총 3회

해설

기울면 가벼운 그룹, 균형이면 저울에 올리지 않은 그룹만 남긴다. 후보 수는 27, 9, 3, 1로 줄어들며 세 번 비교하면 한 개가 남는다.

문제 84

동전의 무게를 저장한 길이 $n=2^k$ 인 배열이 있다($k \geq 1$). 진짜 동전은 모두 같은 무게이며 가짜 하나만 더 가볍다.

매 단계 후보를 같은 크기의 두 절반으로 나누고 가벼운 절반만 남기며, 후보가 1개이면 끝낸다.

실물 양팔저울로는 두 더미를 한 번에 비교한다. 프로그램에서는 두 절반의 무게 합을 모두 배열 원소를 하나씩 읽어 처음부터 계산한다. 원소 하나를 읽고 더하는 비용은 상수 시간이다.

- ① 실물 저울질의 총횟수를 n 으로 나타내시오.
- ② 프로그램이 합산을 위해 읽는 원소 수를 단계별 합으로 쓰고, 전체 합산 시간 복잡도를 구하시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 84 정답과 해설

동전의 무게를 저장한 길이 $n=2^k$ 인 배열이 있다($k \geq 1$). 진짜 동전은 모두 같은 무게이며 가짜 하나만 더 가볍다.

매 단계 후보를 같은 크기의 두 절반으로 나누고 가벼운 절반만 남기며, 후보가 1개이면 끝낸다.

실물 양팔저울로는 두 더미를 한 번에 비교한다. 프로그램에서는 두 절반의 무게 합을 모두 배열 원소를 하나씩 읽어 처음부터 계산한다. 원소 하나를 읽고 더하는 비용은 상수 시간이다.

- ① 실물 저울질의 총횟수를 n 으로 나타내시오.
- ② 프로그램이 합산을 위해 읽는 원소 수를 단계별 합으로 쓰고, 전체 합산 시간 복잡도를 구하시오.

① $\log_2 n$ 회

② $n + n/2 + \dots + 2 = 2n - 2$ 개, 전체 합산 시간 $\Theta(n)$

해설

저울질 한 번마다 후보가 절반이 된다. 프로그램에서는 현재 후보가 m 개일 때 두 절반을 합쳐 m 개의 원소를 읽는다. 후보가 1개인 단계는 이미 끝났으므로 합산하지 않는다. 예를 들어 $n=8$ 이면 $8+4+2=14$ 개를 읽는다.

문제 85

퀵 선택에서 기준값(피벗)보다 작은 원소가 L 개, 같은 원소가 E 개이다. 중복된 값도 각각 세어 전체에서 k 번째로 작은 값을 찾으며 순위는 1부터 센다. $k > L + E$ 여서 큰 값 그룹으로 이동할 때 올바른 새 순위는 $k - L - E$ 이다. $k - L$ 만 사용하면 틀리는 간단한 배열·피벗· k 를 직접 하나 만들고, 그 예에서 올바른 새 순위와 잘못된 새 순위를 비교하여 설명하시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 85 정답과 해설

퀵 선택에서 기준값(피벗)보다 작은 원소가 L 개, 같은 원소가 E 개이다. 중복된 값도 각각 세어 전체에서 k 번째로 작은 값을 찾으려 순위는 1부터 센다. $k > L + E$ 여서 큰 값 그룹으로 이동할 때 올바른 새 순위는 $k - L - E$ 이다. $k - L$ 만 사용하면 틀리는 간단한 배열·피벗· k 를 직접 하나 만들고, 그 예에서 올바른 새 순위와 잘못된 새 순위를 비교하여 설명하시오.

[1,2,2,2,5], 피벗 2, $k=5$ 이면 $L=1, E=3$ 이다. 큰 구간 [5]에서는 1번째를 찾아야 한다.

해설

$k - L = 4$ 는 크기 1인 구간에서 유효하지 않다. 같은 값 E 개도 앞선 순위에 포함된다.

문제 86

퀵 셀렉트로 n 개 원소 중 k 번째로 작은 값을 찾는다. 매 단계 기준값(피벗)을 현재 후보 중에서 모두 같은 확률로 고른다. 무작위 선택 결과를 확률에 따라 평균 낸 실행 시간은 $O(n)$ 이라고 알려져 있다. ① 이것이 모든 실행에서 $O(n)$ 시간을 보장하지 않는 이유를 설명하시오. ② 시간이 많이 걸리게 하는 피벗 선택의 예를 들고, 그때 후보 수가 어떻게 줄어드는지 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 86 정답과 해설

퀵 셀렉트로 n 개 원소 중 k 번째로 작은 값을 찾는다. 매 단계 기준값(피벗)을 현재 후보 중에서 모두 같은 확률로 고른다. 무작위 선택 결과를 확률에 따라 평균 낸 실행 시간은 $O(n)$ 이라고 알려져 있다. ① 이것이 모든 실행에서 $O(n)$ 시간을 보장하지 않는 이유를 설명하시오. ② 시간이 많이 걸리게 하는 피벗 선택의 예를 들고, 그때 후보 수가 어떻게 줄어드는지 쓰시오.

무작위 피벗이 반복해서 극단값이면 후보가 한 개씩만 줄어 이차 시간이 걸린다.

해설

기대값은 피벗 선택의 확률을 평균한 비용이다. 최악 보장과 평균 성능은 별도로 표현한다.

문제 88

삽입 정렬은 먼저 앞의 $n-1$ 개가 정렬되어 있다고 보고, 마지막 원소를 알맞은 위치에 넣어 n 개 정렬을 완성할 수 있다. 그래서 문제 크기가 한 번에 1씩 줄어든다고 표현한다. 서로 다른 n 개 원소가 내림차순인 경우를 들어, 이 사실만으로 전체 시간을 $O(n)$ 이라고 결론낼 수 없는 이유를 설명하시오. 각 삽입에서 앞 원소를 몇 개나 오른쪽으로 밀 수 있는지와 전체 이동 횟수를 근거로 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 88 정답과 해설

삽입 정렬은 먼저 앞의 $n-1$ 개가 정렬되어 있다고 보고, 마지막 원소를 알맞은 위치에 넣어 n 개 정렬을 완성할 수 있다. 그래서 문제 크기가 한 번에 1씩 줄어든다고 표현한다. 서로 다른 n 개 원소가 내림차순인 경우를 들어, 이 사실만으로 전체 시간을 $O(n)$ 이라고 결론낼 수 없는 이유를 설명하시오. 각 삽입에서 앞 원소를 몇 개나 오른쪽으로 밀 수 있는지와 전체 이동 횟수를 근거로 쓰시오.

삽입 단계마다 여러 원소를 옮길 수 있어, 최악 시간은 $\Theta(n^2)$ 이다.

해설

역순 입력의 이동 횟수는 $1+2+\dots+(n-1)$ 이다. 단계별 비용을 더해야 한다. 이미 정렬된 입력에서는 각 삽입이 상수 시간이며, 전체는 $\Theta(n)$ 이다. 정렬된 입력만 선형 시간인 것은 아니다.

문제 89

다음 Python 코드에서 n 은 0 이상의 정수이고, $//$ 는 나눗셈의 정수 몫이다.

```
def f(n):  
    print(n)  
    f(n // 2)
```

$f(4)$

- ① 이 코드가 재귀 호출을 스스로 멈추어 정상적으로 끝나지 못하는 이유를 설명하시오.
- ② 자기 자신을 더 호출하지 않고 정상 종료하려면 어떤 조건과 반환문을 어디에 추가해야 하는지 쓰시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 89 정답과 해설

다음 Python 코드에서 n 은 0 이상의 정수이고, $//$ 는 나눗셈의 정수 몫이다.

```
def f(n):  
    print(n)  
    f(n // 2)
```

$f(4)$

- ① 이 코드가 재귀 호출을 스스로 멈추어 정상적으로 끝나지 못하는 이유를 설명하시오.
- ② 자기 자신을 더 호출하지 않고 정상 종료하려면 어떤 조건과 반환문을 어디에 추가해야 하는지 쓰시오.

- ① 4,2,1,0 뒤에도 $0//2=0$ 이므로 $f(0)$ 을 계속 호출한다.
- ② $f(n//2)$ 를 호출하기 전에 `if n==0: return`을 넣는다.

해설

$n=0$ 에서 더 호출하지 않으면 앞선 호출도 차례로 끝난다. 예를 들어 `print(n)`과 `f(n//2)` 사이에 `if n==0: return`을 넣는다. 함수 시작에 같은 조건을 넣어도 올바르다. 원래 코드는 스스로 멈추지 못해 실제 Python에서는 호출 한도 오류로 중단될 수 있다.

문제 90

N 행 N 열 별무늬를 만드는 함수가 있다. $N=3^k$ 이고 k 는 1 이상의 정수이다. 크기가 1이면 별 하나를 반환하고 끝낸다. 그 외에는 크기 $N/3$ 의 무늬를 같은 함수로 한 번 구해 3×3 구역 중 가운데를 제외한 8개 구역에 재사용한다. ① 최초 호출과 크기 1의 마지막 호출을 포함한 총 호출 횟수를 k 로 나타내시오. ② 별과 공백을 합쳐 N^2 개 문자를 한 글자씩 출력한다면 출력 시간은 어떤 차수인가? 문자당 출력 비용은 일정하고 줄바꿈은 제외한다. ③ 호출 수만으로 전체 시간을 판단할 수 없는 이유를 설명하시오.

풀이 과정과 판단 근거를 함께 쓰시오.

문제 90 정답과 해설

N 행 N 열 별무늬를 만드는 함수가 있다. $N=3^k$ 이고 k 는 1 이상의 정수이다. 크기가 1이면 별 하나를 반환하고 끝낸다. 그 외에는 크기 $N/3$ 의 무늬를 같은 함수로 한 번 구해 3×3 구역 중 가운데를 제외한 8개 구역에 재사용한다. ① 최초 호출과 크기 1의 마지막 호출을 포함한 총 호출 횟수를 k 로 나타내시오. ② 별과 공백을 합쳐 N^2 개 문자를 한 글자씩 출력한다면 출력 시간은 어떤 차수인가? 문자당 출력 비용은 일정하고 줄바꿈은 제외한다. ③ 호출 수만으로 전체 시간을 판단할 수 없는 이유를 설명하시오.

① $k+1$ 회 ② $\Theta(N^2)$ ③ 호출 횟수 외에 무늬 구성과 출력 작업이 필요하다.

해설

호출 크기는 $3^k, 3^{k-1}, \dots, 3, 1$ 이므로 $k+1$ 회이다. 출력할 위치는 N^2 개이고 한 위치마다 일정한 시간이 들어 출력 자체가 $\Theta(N^2)$ 이다. 따라서 재귀 호출이 $\Theta(\log N)$ 회라는 사실만으로 전체 시간이 $O(\log N)$ 이라고 할 수 없다.

문제 91

길이 n 인 정수 배열 a 와 정수 x 가 주어진다. x 가 처음 등장하는 인덱스(0부터)를 반환하고 없으면 -1 을 반환하는 `first_index`를 작성하시오. $0 \leq n \leq 1000$ 이며 원소와 x 는 C의 `int` 범위이다. 배열은 바꾸지 않는다.

Python: `first_index(a,x)` / C: `int first_index(const int a[],int n,int x)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 91 정답과 해설

첫 등장 위치

왼쪽부터 검사하고 첫 일치에서 반환한다.

해설

예: [8,3,8]에서 8을 찾으면 인덱스 0에서 즉시 반환한다.

끝까지 일치하지 않거나 빈 배열이면 -1이다. 배열은 바꾸지 않는다.

최악 시간 $\Theta(n)$, 추가 공간 $\Theta(1)$.

문제 91 Python 구현

첫 등장 위치

```
01 def first_index(a, x):  
02     for i, value in enumerate(a):  
03         if value == x:  
04             return i  
05     return -1
```

시간 $O(n)$.

문제 91 C 구현

첫 등장 위치

```
01 int first_index(const int a[], int n, int x) {  
02     for (int i = 0; i < n; ++i)  
03         if (a[i] == x) return i;  
04     return -1;  
05 }
```

시간 $O(n)$.

문제 92

길이 n 인 배열 a 에는 서로 다른 정수가 오름차순으로 있다. 정수 x 의 인덱스(0부터)를 반환하고 없으면 -1 을 반환하는 `binary_search`를 작성하시오. $0 \leq n \leq 1000$ 이며 원소와 x 는 C의 `int` 범위이다.

Python: `binary_search(a,x)` / C: `int binary_search(const int a[],int n,int x)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 92 정답과 해설

이진 탐색

양 끝을 포함한 탐색 구간을 유지하며 중간값 비교 후 구간을 줄인다.

해설

예: [2,5,8,11,14,17,20]에서 17은 인덱스 3, 5를 검사해 찾는다.

중간값이 작으면 $lo=mid+1$, 크면 $hi=mid-1$ 로 검사한 원소도 제외한다.

$lo>hi$ 이면 -1 . 최악 시간 $O(\log(n+1))$, 추가 공간 $\Theta(1)$.

문제 92 Python 구현

이진 탐색

```
01 def binary_search(a, x):
02     lo, hi = 0, len(a) - 1
03     while lo <= hi:
04         mid = lo + (hi - lo) // 2
05         if a[mid] == x:
06             return mid
07         if a[mid] < x:
08             lo = mid + 1
09         else:
10             hi = mid - 1
11     return -1
```

시간 $O(\log(n+1))$.

문제 92 C 구현

이진 탐색

```
01 int binary_search(const int a[], int n, int x) {
02     int lo = 0, hi = n - 1;
03     while (lo <= hi) {
04         int mid = lo + (hi - lo) / 2;
05         if (a[mid] == x) return mid;
06         if (a[mid] < x) lo = mid + 1;
07         else hi = mid - 1;
08     }
09     return -1;
10 }
```

시간 $O(\log(n+1))$.

문제 95

정수 a, b 의 최대공약수를 반환하는 `gcd_value(a, b)`를 반복문으로 작성하시오.
 $0 \leq a, b \leq 10^9$ 이며 두 값이 모두 0인 경우의 반환값은 0으로 정한다. 둘 중 하나만 0이면 다른 값이 최대공약수이다.

Python: `gcd_value(a, b)` / C: `int gcd_value(int a, int b)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 95 정답과 해설

유클리드 호제법

(a,b) 를 $(b, a \bmod b)$ 로 바꾸다가 b 가 0이면 a 를 반환한다.

해설

예: $(84,30)$, $(30,24)$, $(24,6)$, $(6,0)$ 순서로 바뀌며 답은 6이다.

$b=0$ 이면 나머지를 계산하지 않는다. $(0,0)$ 의 반환값은 약속대로 0이다.

나머지 연산을 $O(1)$ 로 셀 때 시간 $O(\log(\max(a,b)+2))$, 추가 공간 $\Theta(1)$.

문제 95 Python 구현

유클리드 호제법

```
01 def gcd_value(a, b):  
02     while b != 0:  
03         a, b = b, a % b  
04     return a
```

시간 $O(\log(\max(a,b)+2))$.

문제 95 C 구현

유클리드 호제법

```
01 int gcd_value(int a, int b) {  
02     while (b != 0) {  
03         int r = a % b;  
04         a = b;  
05         b = r;  
06     }  
07     return a;  
08 }
```

시간 $O(\log(\max(a,b)+2))$.

문제 97

길이 n 인 정수 배열 a 를 원본 배열 안에서 오름차순으로 바꾸는 `insertion_sort`를 작성하시오. $0 \leq n \leq 1000$ 이며 원소는 C의 `int` 범위이다. 앞 원소는 삽입할 값보다 클 때만 오른쪽으로 민다. 별도 정렬 결과를 반환하지 않는다.

Python: `insertion_sort(a)` / C: `void insertion_sort(int a[], int n)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 97 정답과 해설

삽입 정렬

현재 값을 보관한 뒤 그보다 큰 앞 원소만 한 칸씩 민다.

해설

예: [5,2,4,1]의 앞부분은 [2,5], 다음 [2,4,5]로 정렬된다.
key보다 큰 값만 옮기므로 같은 값의 원래 순서를 유지한다.
최선 $\Theta(n)$, 최악 $\Theta(n^2)$, 추가 공간 $\Theta(1)$. 원본 배열을 바꾼다.

문제 97 Python 구현

삽입 정렬

```
01 def insertion_sort(a):
02     for i in range(1, len(a)):
03         key = a[i]
04         j = i - 1
05         while j >= 0 and a[j] > key:
06             a[j + 1] = a[j]
07             j -= 1
08         a[j + 1] = key
```

최악 시간 $O(n^2)$, 정렬된 입력 $O(n)$.

문제 97 C 구현

삽입 정렬

```
01 void insertion_sort(int a[], int n) {  
02     for (int i = 1; i < n; ++i) {  
03         int key = a[i], j = i - 1;  
04         while (j >= 0 && a[j] > key) {  
05             a[j + 1] = a[j];  
06             --j;  
07         }  
08         a[j + 1] = key;  
09     }  
10 }
```

최악 시간 $O(n^2)$, 정렬된 입력 $O(n)$.

문제 98

길이 n 인 정수 배열 a 에서 중복도 각각 세어 k 번째로 작은 값을 반환하는 `quick_select`를 작성하시오. $1 \leq n \leq 1000$, $1 \leq k \leq n$ 이며 원소는 C의 `int` 범위이다. k 는 1부터 세는 순위이고 배열 변경을 허용한다.

Python: `quick_select(a,k)` / C: `int quick_select(int a[],int n,int k)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 98 정답과 해설

퀵 선택

피벗 이하를 앞으로 모으고 피벗의 최종 위치와 목표 인덱스를 비교한다.

해설

목표 인덱스는 $k-1$ 로 고정한다. 피벗의 위치 p 와 비교해 한쪽만 남긴다.
같은 값도 각각 순위에 포함한다. 이 구현은 마지막 원소를 피벗으로 쓴다.
최악 $\Theta(n^2)$, 추가 공간 $\Theta(1)$. 무작위 피벗의 기대 시간 보장과 구별한다.

문제 98 Python 구현

퀵 선택트 (왼쪽 코드 다음에 오른쪽 코드가 이어짐)

```
01 def quick_select(a, k):
02     lo, hi, target = 0, len(a) - 1, k - 1
03     while lo <= hi:
04         pivot, p = a[hi], lo
05         for j in range(lo, hi):
06             if a[j] <= pivot:
07                 a[p], a[j] = a[j], a[p]
08                 p += 1
09         a[p], a[hi] = a[hi], a[p]
10         if p == target:
11             return a[p]
12         if p < target:
13             lo = p + 1
14         else:
15             hi = p - 1
```

이 고정 피벗 구현의 최악 시간 $O(n^2)$. 모든 입력에서 선형이라고 주장할 수 없다.

문제 98 C 구현

퀵 선택트 (왼쪽 코드 다음에 오른쪽 코드가 이어짐)

```
01 int quick_select(int a[], int n, int k) {
02     int lo=0, hi=n-1, target=k-1;
03     while (lo <= hi) {
04         int pivot=a[hi], p=lo;
05         for (int j=lo; j<hi; ++j) {
06             if (a[j] <= pivot) {
07                 int t=a[p]; a[p++]=a[j]; a[j]=t;
08             }
09     }

10     int t=a[p]; a[p]=a[hi]; a[hi]=t;
11     if (p == target) return a[p];
12     if (p < target) lo=p+1;
13     else hi=p-1;
14 }
15 return 0; /* unreachable for valid k */
16 }
```

이 고정 피벗 구현의 최악 시간 $O(n^2)$. 모든 입력에서 선형이라고 주장할 수 없다.

문제 99

피보나치 수는 $F(0)=0$, $F(1)=1$, $n \geq 2$ 에서 $F(n)=F(n-1)+F(n-2)$ 이다. 정수 $0 \leq n \leq 90$ 에 대해 $F(n)$ 을 반환하는 `fibonacci(n)`을 반복문으로 작성하시오. C 반환형은 `long long`이다. 필요한 $F(n)$ 까지만 계산하시오.

Python: `fibonacci(n)` / C: `long long fibonacci(int n)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 99 정답과 해설

반복 피보나치

$n \geq 2$ 에서 $F(2)$ 부터 $F(n)$ 까지만 차례로 계산한다.

해설

$n=0$ 이면 0, $n=1$ 이면 1을 반환한다. 이후 두 이전 항의 합을 구한다.

$F(90)=2880067194370816120$. 필요한 $F(n)$ 까지만 계산한다.

기본 덧셈을 $O(1)$ 로 셀 때 시간 $\Theta(n)$, 추가 공간 $\Theta(1)$.

문제 99 Python 구현

반복 피보나치

```
01 def fibonacci(n):  
02     if n < 2:  
03         return n  
04     a, b = 0, 1  
05     for i in range(2, n + 1):  
06         a, b = b, a + b  
07     return b
```

시간 $O(n)$. $F(90)=2880067194370816120$ 이다.

문제 99 C 구현

반복 피보나치

```
01 long long fibonacci(int n) {  
02     if (n < 2) return n;  
03     long long a=0, b=1;  
04     for (int i=2; i<=n; ++i) {  
05         long long next=a+b;  
06         a=b; b=next;  
07     }  
08     return b;  
09 }
```

시간 $O(n)$. $F(90)=2880067194370816120$ 이다.

문제 100

길이 $n=2^k$ ($1 \leq n \leq 1024$, k 는 0 이상의 정수)인 배열 `weights`에 0이 정확히 하나 있고 나머지는 1이다. 왼쪽 절반의 합을 조사하며 0의 인덱스(0부터)를 반환하는 `light_coin`을 작성하시오.

Python: `light_coin(weights)` / C: `int light_coin(const int weights[], int n)`

Python 또는 C 하나로 함수와 시간 복잡도를 작성하시오. 기본 정수 연산은 $O(1)$ 로 센다.

문제 100 정답과 해설

가벼운 동전 찾기

왼쪽 합이 원소 수보다 작으면 왼쪽, 같으면 오른쪽으로 범위를 줄인다.

해설

예: [1,1,0,1]에서 왼쪽 합은 2이므로 오른쪽 두 원소만 남긴다.

그다음 왼쪽 합이 0이면 그 위치가 답이다. 한 개가 남으면 종료한다.

합산량 $n/2+n/4+\dots+1$. 시간 $\Theta(n)$, 추가 공간 $\Theta(1)$.

문제 100 Python 구현

가벼운 동전 찾기

```
01 def light_coin(weights):
02     lo, hi = 0, len(weights)
03     while hi - lo > 1:
04         mid = (lo + hi) // 2
05         total = sum(weights[i] for i in range(lo, mid))
06         if total < mid - lo:
07             hi = mid
08         else:
09             lo = mid
10     return lo
```

배열 합 연산을 포함한 시간 $O(n)$. 범위를 줄이는 단계 수만 $O(\log n)$ 이다.

문제 100 C 구현

가벼운 동전 찾기

```
01 int light_coin(const int weights[], int n) {
02     int lo=0, hi=n;
03     while (hi-lo > 1) {
04         int mid=(lo+hi)/2, total=0;
05         for (int i=lo; i<mid; ++i) total+=weights[i];
06         if (total < mid-lo) hi=mid;
07         else lo=mid;
08     }
09     return lo;
10 }
```

배열 합 연산을 포함한 시간 $O(n)$. 범위를 줄이는 단계 수만 $O(\log n)$ 이다.